WHEN DOES KV REUSE WORK FOR AGENT MEMORY?

A CONTROLLED STUDY OF PAYLOAD FIDELITY, STORE CONDITIONING, AND CACHE HARVESTING—WITH A REPRODUCIBLE STATE-OF-THE-ART SYSTEM

Zefeng Cai
3294005475a@gmail.com

ABSTRACT

Long-horizon agents accumulate interaction histories that outgrow the context window and must be offloaded to an external memory. Most memory systems hand the reader what they constructed—facts, graphs, summaries—even when they also keep the raw history, and constructions paraphrase away the exact values and steps that dense agent trajectories are queried for; a growing line of work instead serves the raw history from a cheap index. What is missing is measurement: *which* property of raw payloads carries the benefit, and *when* the trajectory's own KV cache can stand in for re-encoded text. This paper is a controlled study of both questions, shipped with a reproducible system.

On payload form, matched-evidence ablations on dense agent trajectories—the same cited evidence presented as verbatim spans, original-line extracts, LLM summaries, or atomic facts—show that *generative paraphrase* of matched evidence costs 8–$14\,\mathrm{pp}$, and that in the deployed pipeline deterministic extractive shortening costs about half of what generative re-encoding does, extending controlled memory-representation comparisons to agent traces and separating paraphrase damage from compression damage. On KV reuse, we distinguish computational fidelity from task utility. Within the window, gathering the selected spans' cached KV at their original positions is computationally faithful to a full-prefill-then-mask oracle and *accuracy-equivalent* to re-prefilling the same spans as text—we claim a cost property, not an accuracy advantage: first-token latency decouples from evidence size and beats a resident prefix-cache baseline. Beyond the window, where no joint prefill exists, independently encoded event stores collapse on the official benchmark ($-13\,\mathrm{pp}$ paired), two known repair families recover 72%, and two scoped regularities organize the rest: evidence converts to accuracy only through joint encoding, and a store's write-time conditioning must match what is served, in content and in position—in a six-way selector comparison for the fresh-recompute budget, no content signal, including a KEEP-style attention-propagation selector, beats recency. Where the agent's own rollout cache survives, harvesting it replaces the $5\times$ store-encoding cost at parity (8B/32B, Qwen/Llama, AMA-Bench/LOCOMO), and a mixed deployment policy matches the routed-text carrier baseline overall—the gain concentrated in the in-window harvest slice, the over-window gap left open and stated.

The findings ship as VeraKV, a verbatim-first memory stack over self-hosted inference: ranked #1 on the AMA-Bench leaderboard as of 2026-07-15 and competitive on LOCOMO and LongMemEval-S, with all code, configurations, prompts, predictions, and judge outputs released.[1]

1 INTRODUCTION

A long-horizon agent's state is its history: the tool calls it issued, the observations it received, the reasoning it wrote. As that history grows past the model's context window, the agent needs a memory. The prevailing answer is to *construct* one—to run an LLM over the trajectory that extracts

[1] https://github.com/oklen/VeraKV

atomic facts (Mem0 (Chhikara et al., 2025)), writes and links notes (A-Mem (Xu et al., 2025)), distills hierarchical summaries (MemoryBank (Zhong et al., 2024)), or builds a temporal knowledge graph (Zep (Rasmussen et al., 2025))—and then retrieve over what was constructed. Many of these systems keep the raw history as well (Mem0, for one, stores the original messages); what matters is what reaches the reader. Retrieval returns the constructed units—Mem0 returns only its extracted facts—so which details the reader can ever see is fixed at write time, before any question exists. This is natural for *dialogue* memory, where the salient content is a handful of stable user facts. It is a poor fit for *agent trajectories*, where the answer is often an exact value or a specific step that a fact-extractor paraphrases away. AMA-Bench (Zhao et al., 2026) makes this concrete: purpose-built dialogue-memory systems (Mem0 0.21, A-Mem 0.32, MemGPT 0.33, MemoryBank 0.34) all score *below* a no-memory long-context baseline (0.52) on dense agent trajectories, and the benchmark's own needle control locates two losses: serving MemoryBank's constructed memory of the answer-bearing turns instead of the turns themselves drops accuracy from 0.46 to 0.27 (-41% relative, BabyAI), and similarity retrieval then misses those turns (HippoRAG2 loses a further 43% end-to-end). What reaches the reader—not whether the raw history was kept—is where these systems lose.

A second line of work already draws the natural conclusion and serves the raw history from a cheap index: gist navigation with raw-passage lookup (ReadAgent (Lee et al., 2024)), full-fidelity trace databases indexed by summaries (Memex (Wang et al., 2026), HORMA (Hsu et al., 2026)), budgeted verbatim evidence capsules (EMBER (Li et al., 2026c)), optical indexes with deterministic verbatim return (OCR-Memory (OCR-Memory authors, 2026)), and—concurrently with this work—a controlled comparison showing verbatim chunks beat extracted artifacts on dialogue benchmarks by 16–$22\,\mathrm{pp}$ (An, 2026). "Serve the raw text" is no longer a contrarian stance; it is a design pattern. What the pattern still lacks is *attribution* and a *cost map*. Which property of the raw payload carries the benefit—and is it paraphrase or mere shortening that does the damage? The reader consuming the memory is a separate axis that end-to-end comparisons leave entangled with the store (MemDelta authors, 2026); how much of a system gap is reader prompting? And since the trajectory was prefilled once anyway on self-hosted inference (vLLM (Kwon et al., 2023), SGLang (Zheng et al., 2024)), when can the resulting KV cache—the model's own reading of the evidence—be *served* in place of re-encoded text, and when does that reuse break? Throughout we keep three notions separate that are easy to conflate: *source fidelity* (is the payload's text unaltered?), *computational fidelity* (does served KV reproduce the full-prefill computation?), and *task utility* (does the answer come out right?)—the first two never imply the third, and we measure each on its own terms. Source fidelity is about *tokens*, not information: a served row's keys and values are verbatim those of the original tokens, but because the row was contextualized during prefill it can carry causal influence from tokens the query never selected (§5). We therefore say the memory serves selected *rows*, not selected *information*, and are careful not to conflate the two.

We answer these questions with controlled experiments organized around VeraKV, the verbatim-first memory stack we build and release (§2); the stack is the instrument and the artifact, not a claim of a new memory primitive. Concretely: i) *payload-form attribution on dense agent trajectories*: matched-evidence ablations in which only the payload form varies show that re-encoding the *same* selected evidence generatively costs 8–$14\,\mathrm{pp}$ while, in the deployed pipeline, a deterministic swap to its original lines costs about half as much as a generative re-encoding (-2.6 vs. $-5.1/-5.9\,\mathrm{pp}$)—extending controlled memory-representation comparisons (An, 2026) from dialogue to agent traces and separating *paraphrase* damage from *compression* damage, with the effect largest where answers are exact (§4.1, §4.4); ii) *reader–memory separation as measurement discipline*: holding the memory fixed, a one-sentence answer-instruction swap moves accuracy more than any of fourteen reader-side mechanisms, and the answering protocol can travel inside the memory's return value—applying concurrent skill-KV transplantation work (Li, 2026) to the agent-memory answering protocol at text-instruction parity, then extended to a query-routed skill *library* in which additive per-class hints add a further $+6.3\,\mathrm{pp}$ on routed questions while replacement routing regresses (§4.4, §4.6); iii) *a validated within-window KV serving implementation*: position-preserving gather of selected spans reproduces a full-prefill-then-mask oracle (under stated cache/position assumptions) and is *accuracy-equivalent* to re-prefilling the same spans as text—the benefit we claim is cost, not accuracy: per-query prefill decouples from evidence size, beating text re-prefill and a resident prefix-cache baseline on TTFT (§4.5, §4.7); iv) *an out-of-window characterization*—the study's core: independently encoded event stores collapse on the official harness ($-13.2\,\mathrm{pp}$ paired, $n{=}2431$), repair families adapted from RAG-oriented cache splicing recover 72% but their parity claims do not transport, two scoped reg-

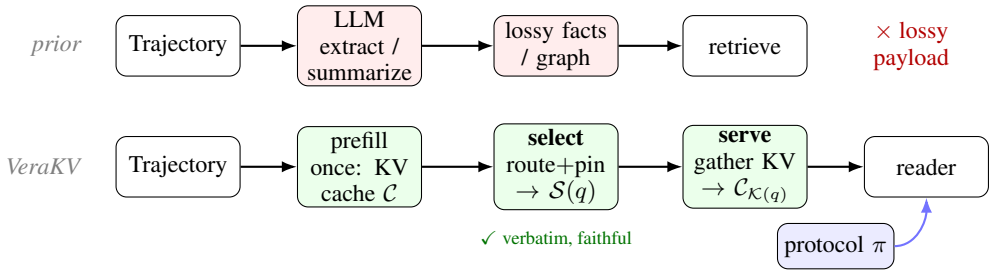


Figure 1: Prior memory serves the reader what it *constructed* at write time—a lossy payload, even when the raw history is also kept. VeraKV prefills the trajectory once into a KV cache, then per query **selects** which spans to keep ($\mathcal{S}(q)$) and **serves** them verbatim by gathering their cached KV at original positions ($\mathcal{C}_{\mathcal{K}(q)}$); an optional answering protocol π rides at the front of the returned context. Select decides *which* evidence, KV decides *how faithfully* it is carried.

ularities organize the failure (evidence converts to accuracy only through joint encoding; write-time conditioning must match serving in content and position), no content-signal selector in a six-way comparison—including a KEEP-style attention-propagation selector (Yang et al., 2026)—beats recency for the fresh-recompute budget, and where the agent's rollout cache survives, harvesting it replaces the $5\times$ store-encoding cost at parity across scales (8B/32B), benchmark families (AMA-Bench/LOCOMO), and backbones (Qwen/Llama); a mixed deployment policy then matches the routed-text carrier baseline overall, its gain concentrated in the in-window harvest slice, the over-window gap left open (§5); and v) *a reproducible system*: code, configurations, prompts, raw predictions, and judge outputs for every reported run; the memory stack alone (fixed default reader) already matches the purpose-built AMA-Agent, and the full disclosed system is ranked #1 on the public AMA-Bench leaderboard as of 2026-07-15, while remaining competitive on LOCOMO and LongMemEval-S (§4.1, §4.2, §4.3).

2 Method: VeraKV

VeraKV is a KV-cache memory manager for self-hosted long-horizon agents. It realizes one principle—deliver the evidence faithfully, never construct a lossy store—through two operations: *select* the raw spans a query needs, and *serve* them verbatim. Selection is one layer; *serving* splits into three regimes that this paper keeps explicitly distinct (Table 1): *text rehydration* (general; produces every headline accuracy number), *whole-trajectory KV gather* (exact, only while the trajectory fits the positional window), and *per-event anchored KV stores* (the over-window regime, approximate by necessity; characterized in §5). When a result below depends on which regime served it, we say so at the point of use, and Appendix A maps every reported number to its pipeline. We make the pieces precise below (Fig. 1, Algorithm 1, Table 2).

Setup. A trajectory is a sequence of segments (turns) $T = (s_1, \ldots, s_n)$, where s_i carries a token string x_i; write $x = x_1 \| \cdots \| x_n$ for their concatenation, $N = \sum_i |x_i|$, and $\mathrm{span}(i) = [\ell_i, r_i)$ for the token interval of s_i in x ($\ell_i = \sum_{j<i} |x_j|$). The reader f_θ is a frozen autoregressive transformer with rotary position embeddings (RoPE). Building the memory prefills x *once* into a key–value cache

$$\mathcal{C} = \{(k_t, v_t)\}_{t=1}^{N}, \qquad (k_t, v_t) = f_\theta\text{'s key/value at position } t \text{ attending to all } t' \le t,$$

and attaches to each cold segment an extractive gist $g_i = \textsc{Head}(s_i)$ (a prefix of its action/observation, not an LLM summary). The memory is $M = (\{s_i\}, \{g_i\}, \mathcal{C}, \{\mathrm{span}(i)\})$: the gist is a routing index and a reader-facing navigation scaffold, but the *authoritative* evidence—what any query is answered from—is always the raw x_i.

Select: which spans a query needs. Given a query q and a token budget B, retrieval forms three tiers. The *hot* set is the last h turns, $H = \{n-h+1, \ldots, n\}$. The *pin* set is the turns q names by a structural address—"step N", "turns N–M"—dereferenced deterministically, $P(q) = \{i : \mathrm{addr}(q) \ni i\}$. The remaining cold turns $\mathcal{I}_{\mathrm{cold}}$ are scored by a pluggable router $R(q, s_i)$ and taken

greedily under budget, pins ranked first:

$$\mathcal{S}(q) = \text{top-}k_{\,i \in P(q) \cup \mathcal{I}_{\text{cold}}}\, R(q, s_i) \quad \text{s.t.} \quad \sum_{i \in \mathcal{S}(q)} |x_i| \le B, \qquad \mathcal{K}(q) = H \cup \mathcal{S}(q),$$

with $\mathcal{K}(q)$ kept in temporal order. R ranges over lexical overlap, dense cosine (with a lexical prefilter), and their reciprocal-rank fusion; the pin is a structural cue, not a scorer. Agent trajectories are event logs with stable addresses (step indices, tool-call ids, file paths), and 82% of AMA-Bench questions carry at least one (step indices alone in 73%), so the memory dereferences a named address deterministically rather than forcing all access through similarity. How much the router matters is regime-dependent: on step-indexed traces a cheap router saturates, while on diffuse dialogue it carries more of the work (§4.4).

Serve: verbatim spans, natively as KV. The selected spans are served verbatim, and their native form is the cache. Rather than re-encode them, we *gather* their cached key–values at the original positions,

$$\mathcal{C}_{\mathcal{K}(q)} = \Big\{ (k_t, v_t) \in \mathcal{C} \;:\; t \in \textstyle\bigcup_{i \in \mathcal{K}(q)} \text{span}(i) \Big\},$$

and decode the answer by re-prefilling only q on top of $\mathcal{C}_{\mathcal{K}(q)}$. This gather is *exact*: because RoPE makes attention depend only on the relative offset $t - t'$, keeping each retained token at its original position reproduces a full-prefill-then-mask oracle,

$$f_\theta\big(\cdot \mid \mathcal{C}_{\mathcal{K}(q)},\, q\big) \;=\; f_\theta^{\text{mask}}\big(\cdot \mid x,\, \mathcal{K}(q),\, q\big),$$

where f_θ^{mask} prefills all of x but masks the decode's attention to the retained positions $\bigcup_{i \in \mathcal{K}(q)} \text{span}(i)$. Crucially each gathered (k_t, v_t) was computed while attending to earlier *unselected* turns, so it carries trajectory-conditioned context that a compact re-prefill of x_i in isolation cannot reconstruct—§4.4 measures the residual to the oracle at ~ 0.7 logits, versus ~ 13.5 for the isolated re-prefill. The per-query prefill is therefore $O(|q|)$, independent of the evidence size $\sum_{i \in \mathcal{S}(q)} |x_i|$, which the text path re-prefills in full (§4.7). When a single observation overflows the budget, a query-focused filter keeps only its query-relevant *lines*, verbatim; retained evidence is always verbatim, even if not every line of an oversized turn is retained.

The answering protocol travels in the return value. `retrieve()` returns a context the reader consumes, so it can carry more than evidence. VeraKV optionally prepends a fixed, query-independent *answering protocol* π to that context—framed as a memory-service note, with the host reader's instruction untouched. As a fixed prefix, π is prefilled once at position 0 and reused, and it recovers most of a reader-side instruction's benefit from inside the memory (§4.4). It is off by default.

Assembly. Two modes share the store. The *text pipeline*, which produces all headline numbers, returns

$$C(q) = \big[\, \pi \parallel O \parallel (x_i)_{i \in \mathcal{S}(q)} \parallel q \,\big], \qquad a \sim f_\theta\big(C(q)\big),$$

where the overview O is the hot turns verbatim plus every cold turn's gist and is prefilled once per episode. The *KV-serving* mode drops O and serves $[(x_i)_{i \in \mathcal{K}(q)} \| q]$ through $\mathcal{C}_{\mathcal{K}(q)}$. Both keep each turn's explicit `<step N>` tag, so temporal order is read from tags, not physical position.

Scope of the cache-native path. KV gather is exact only when the source turns were prefilled within the model's positional window; the text path carries no such assumption and is the general path, in particular for the over-window trajectories this paper targets—AMA-Bench SOFTWARE runs 60–100k tokens against a 22k budget. Table 1 states the three regimes: text rehydration produces every headline number, full-trajectory KV gather is exact only within the window (the §4.7 prototype), and chunked KV gather is a systems extension.

3 EXPERIMENTAL SETUP

We evaluate on three long-term-memory families that stress different inputs: **AMA-Bench** (Zhao et al., 2026) (agent trajectories), **LOCOMO** (Maharana et al., 2024) (multi-session dialogue), and

Table 1: Three ways to serve the selected evidence, and their scope. Only text rehydration produces this paper's accuracy numbers; cache-native serving is a within-window prototype.

Serving mode	Over-window?	Global traj.-cond. KV?	Headline?
Text rehydration	yes	— (re-encoded)	**yes**
Full-trajectory KV gather	only if it fits	yes (exact)	prototype
Chunked KV gather	yes	chunk-local only	future

Algorithm 1 VeraKV: build once, then select-and-serve per query. The gather step returns the full-prefill KV of the retained positions, so the KV decode reproduces f_θ^{mask} (§2) while re-prefilling only q; the text pipeline assembles the same selection with an overview.

procedure BUILDMEMORY($T = (s_1, \ldots, s_n)$)
  $x \leftarrow x_1 \| \cdots \| x_n$; record $\text{span}(i) = [\ell_i, r_i)$ for each s_i
  $g_i \leftarrow \text{HEAD}(s_i)$ for each s_i ▷ extractive gist; index only, never payload
  if $N \leq$ positional window **then**
    $\mathcal{C} \leftarrow \text{PREFILL}_{f_\theta}(x)$ ▷ one joint prefill: exact KV serving available
  else
    $\mathcal{C} \leftarrow \{\text{ENCODEANCHORED}_{f_\theta}(s_i)\}_{i=1}^{n}$ ▷ per-event anchored stores, approximate (§5); text path needs no $\mathcal{C}$
  end if
  return $M = (\{s_i\}, \{g_i\}, \mathcal{C}, \{\text{span}(i)\})$
end procedure

procedure RETRIEVE($M,\ q,\ B$)
  $H \leftarrow \{n{-}h{+}1, \ldots, n\}$ ▷ hot: last h turns
  $P(q) \leftarrow \{i : \text{addr}(q) \ni i\}$ ▷ pin: structural addresses in q
  $\mathcal{S}(q) \leftarrow \text{top-}k_{\,i \in P(q) \cup \mathcal{I}_{\text{cold}}} R(q, s_i)$ s.t. $\sum_{i \in \mathcal{S}(q)} |x_i| \leq B$
  $\mathcal{K}(q) \leftarrow H \cup \mathcal{S}(q)$ ▷ keep in temporal order
  if KV-serving **then**
    $\mathcal{C}_{\mathcal{K}(q)} \leftarrow \{(k_t, v_t) \in \mathcal{C} : t \in \bigcup_{i \in \mathcal{K}(q)} \text{span}(i)\}$ ▷ gather at original positions
    return $f_\theta(\cdot \mid \mathcal{C}_{\mathcal{K}(q)},\ q)$ ▷ re-prefill only q
  else
    $O \leftarrow [\,\text{hot verbatim} \| (g_i)_{i \in \mathcal{I}_{\text{cold}}}\,]$ ▷ overview, per-episode
    return $f_\theta\big(\cdot \mid [\,\pi \| O \| (x_i)_{i \in \mathcal{S}(q)} \| q\,]\big)$
  end if
end procedure

LongMemEval-S (Wu et al., 2025) (long-context chat). We follow each benchmark's official protocol—same reader and judge, its own metric. For AMA-Bench this is the official `run.py` harness on the full open-ended set, backbone and judge both Qwen3-32B (Qwen Team, 2025) ("Track B"), 22k context, so our numbers drop into the published leaderboard; unless noted, every AMA-Bench result uses this default and every ablation contrast is same-batch.[2]

Statistical inference. Questions within an episode share a trajectory and are not independent, so every interval we report is an *episode-clustered* bootstrap CI (episodes resampled as clusters; 208 on the full AMA set), and paired McNemar discordant counts are reported as descriptive effect summaries alongside those CIs rather than as stand-alone tests. Contrasts fall in three classes, labeled at the point of use: *confirmatory* (decision rule fixed before the run, e.g. the deployment-map evaluation of §5), *exploratory* (developed iteratively on the public set, e.g. the leaderboard configuration), and *post-hoc mechanism analyses* (the §5 dials); no multiple-testing correction is applied to the exploratory class, and we rely on replication—across scales, benchmark families, and backbones—rather than corrected p-values for the mechanism claims. Appendix A maps every headline number to its pipeline, prompt, sample size, and batch.

[2]The open-ended set is 208 episodes / 2,496 QA over six domains. LOCOMO and LongMemEval-S reader/judge protocols are stated where we report them.

Table 2: VeraKV default configuration, organized by its two pillars. Every row is a swap point. The answer instruction is a separately ablated reader-side variable (App. B), not part of the memory.

Component	Default setting
Store (build once)	
Hot tier	h=4 most recent turns, verbatim
Cold gist g_i	extractive head `action[:120]+obs[:160]` (no LLM)
Prefill $\mathcal{C}$	whole trajectory once into a KV cache
Select $\mathcal{S}(q)$ (per query)	
Router R	lexical / embedding (Qwen3-Emb-0.6B) / RRF hybrid
Step-pin $P(q)$	regex `(turn|step)s?` N `(--`M`)?`; deref named turns
Budget B, top-k	22k tokens (32k window − thinking reserve); k=5
Serve $\mathcal{C}_{\mathcal{K}(q)}$	
Rehydration	selected spans verbatim; KV gathered at original positions
Reader	
Answer prompt	host default; optional protocol π (off by default)
Reader / judge	Qwen3-32B (AMA Track B; LOCOMO/LME: §3)

4 RESULTS

4.1 AMA-BENCH: LEADING RESULTS ON AGENT-TRAJECTORY MEMORY

Table 3 places VeraKV on AMA-Bench. With only a *free lexical* router—no learned index, no construction—VeraKV already matches the purpose-built AMA-Agent (0.5621 vs 0.5722) and beats every other published memory system by $+10$ to $+35$ pp. The deployed router and a structural step-pin, memory-side only, lift this to 0.5954, still above AMA-Agent with no reader-side change; additionally swapping the reader's one-sentence answer instruction for a structured one—a disclosed *reader-side* variable, orthogonal to the memory—reaches **0.6478**, the **#1** entry on the public leaderboard as of 2026-07-15[3], winning *five of six* domains (Table 4). One methodological caveat we state rather than bury: AMA-Bench exposes only a public test split, and the champion configuration was reached by iterative development on it; we therefore label the leaderboard number an *exploratory engineering result* rather than an unbiased generalization estimate. The paired mechanism claims of §4.4 and §5 do not inherit this risk in the same way: their contrasts were pre-registered with fixed decision rules before each run, and the out-of-window recipes were developed entirely on a 103-episode mechanism subset before being evaluated once on the full official set. Where the gains come from is the subject of §4.4.[4]

The one domain VeraKV does not lead, SOFTWARE, is not a retrieval or reasoning gap: named steps are recalled in full and SOFTWARE *reasoning* is in fact VeraKV's strongest question type at budget. The residual is the temporal-scan questions—"which step first/last did X", 19% of the domain—on which every configuration we ran scores low and whose gold labels often do not match the trajectory itself, so we do not attribute the gap to coverage or to AMA-Agent's construction-time state summary.[5]

4.2 LOCOMO: COMPETITIVE WITH DEDICATED DIALOGUE-MEMORY SYSTEMS

On LOCOMO's public protocol—reader and judge both gpt-4o-mini, the Mem0/Zep convention—VeraKV's compact hybrid-retrieval arm reaches $J = 0.704$, beating Mem0 (0.671) and Zep (0.660). Under a single consistent reader/judge that removes the cross-paper confound, compact selection *edges out* full-history prompting ($0.716 > 0.661$): dropping the irrelevant turns denoises, and verbatim memory abstains correctly on the adversarial unanswerable category 87–89% of the time

[3] A later entry (0.6975, dated 2026-09-09) has since overtaken it; as of 2026-09-27 VeraKV ranks second.

[4] The 0.5954 memory-only run carries a 95% episode-clustered bootstrap CI of $[0.570, 0.620]$ (208 clusters; the submitted 0.6478: $[0.622, 0.673]$). Answer instructions are reproduced verbatim in Appendix B; the ranking survives a cross-family Llama-3.1-70B re-judge that scores our predictions *higher* than the same-family judge (Appendix I); and we release all configs, raw predictions, and judge outputs.

[5] The SOFTWARE mechanism is dissected in Appendix E.

Table 3: AMA-Bench, average accuracy over the full 2,496 QA. Baseline rows are the published AMA-Bench numbers; VeraKV rows are our runs, labeled with the answer prompt they use so memory- and reader-side effects stay separable. The 0.6478 run is the submitted entry, verified #1 on the public leaderboard (as of 2026-07-15). Its deployed router fuses lexical overlap with an audited-neutral model-pick signal; we release the reproducible lexical+pin configuration, which our local judge scores at 0.6466 (App. F).

Method	Avg. accuracy
VeraKV (deployed router+pin memory, structured reader)	**0.6478**
HybridFocus (public leaderboard entry; code unreleased)	0.6246
VeraKV (deployed router+pin memory, *default* reader)	0.5954
AMA-Agent (purpose-built)	0.5722
VeraKV (lexical router, default reader)	0.5621
MemoRAG (Qian et al., 2024)	0.4606
HippoRAG2 (Gutiérrez et al., 2024)	0.4480
BM25	0.3436
MemoryBank (Zhong et al., 2024)	0.3397
MemGPT (Packer et al., 2023)	0.3304
A-Mem (Xu et al., 2025)	0.3186
Mem0 (Chhikara et al., 2025)	0.2104
Long-context Qwen3-32B (no memory)	0.52

Table 4: AMA-Bench per-domain accuracy, exactly as listed on the public leaderboard for the 0.6478-verified VeraKV entry, vs the purpose-built AMA-Agent. VeraKV wins five of six domains, losing only SOFTWARE.

	TEXT2SQL	SOFTWARE	WEB	GAME	EMBODIED	OPENWORLD
VeraKV	**0.691**	0.465	**0.694**	**0.706**	**0.636**	**0.700**
AMA-Agent	0.609	**0.636**	0.517	0.644	0.499	0.471

versus 75% for full context, because it knows whether evidence exists. So under a fixed protocol, compact verbatim selection is competitive with dedicated dialogue-memory systems—on *dialogue*, the workload construction was designed for.[6]

4.3 LONGMEMEVAL-S: ROUTED SELECTION BEATS TRUNCATION UNDER BUDGET PRESSURE

LongMemEval-S puts each question over its own ∼115k-token, ∼50-session haystack; we evaluate the *full* 500 questions (470 answerable + 30 abstention), with 95% bootstrap CIs, on the official `longmemeval_s` release.[7] In the Qwen3-32B regime two facts hold. (i) When the history *exceeds* the window—the operative case for long-horizon agents—the full arm must truncate, and routed selection at a fraction of the budget beats it substantially: $0.496\,[0.453, 0.543]$ vs $0.302\,[0.262, 0.343]$ at a 28k budget ($+0.194\,[0.149, 0.238]$). The mechanism is clean: truncation drops old evidence sessions (95% of its answerable misses are *selection* failures), whereas the router keeps the gold session in budget (9% selection-fail). (ii) When the history instead *fits* an extended 128k window (via YaRN), full context wins overall ($0.564\,[0.519, 0.609]$ vs 0.496; $+0.068\,[0.019, 0.119]$)—but at $4.5\times$ the context, and its edge is confined to *recall/breadth*: full leads on single-session recall (ss-user 0.828 vs 0.609, ss-asst 0.946 vs 0.768) and multi-session (0.405 vs 0.322), while routed selection *wins* temporal-reasoning (0.331 vs 0.291) and knowledge-update (0.736 vs 0.708; Fig. 2). So "selection $\geq$ full" is *regime-dependent*. Here the router *is* load-bearing (unlike on AMA): the dense-embedding hybrid beats a lexical router (0.496 vs 0.453; $+0.043\,[0.006, 0.079]$), entirely by cutting selection failures (9% vs 23%).

[6] Under the gpt-4o-mini judge, full-history prompting scores 0.756, and LOCOMO has known protocol variants under which newer file/tool systems report higher; the judge disagreement and these caveats are in Appendix J.

[7] xiaowu0162/longmemeval; the oracle evidence-session labels are used *only* for error attribution between selection and reader failures, never by the method; the exact file and access date ship with our artifacts.

Table 5: LOCOMO. Left: public gpt-4o-mini protocol (comparable to Mem0/Zep). Right: consistent Qwen3-32B judge. "Full-history" prompts the entire dialogue (distractors included); it is a strong baseline, not a gold-evidence oracle.

Public (gpt-4o-mini)		Consistent judge (Qwen3-32B)	
System	J	Arm	J
Full-history prompt	0.756	VeraKV (hybrid retrieval)	**0.716**
VeraKV (hybrid retrieval)	**0.704**	Full-history prompt	0.661
Mem0 (Chhikara et al., 2025)	0.671	VeraKV (lexical retrieval)	0.590
Zep (Rasmussen et al., 2025)	0.660		

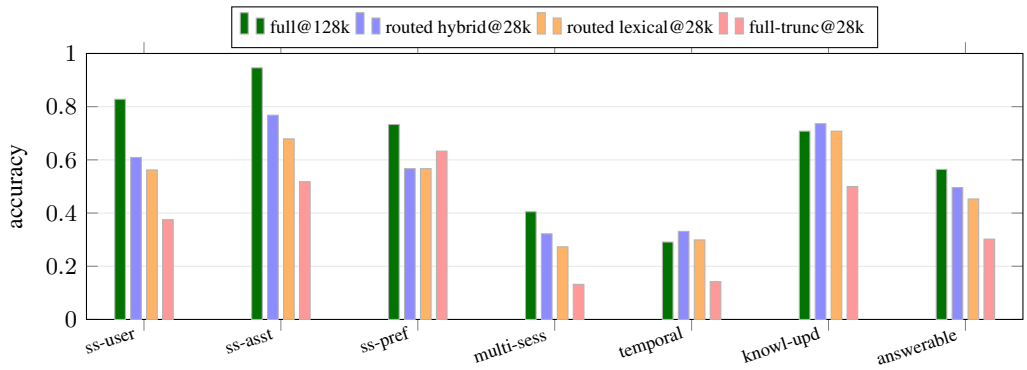


Figure 2: LongMemEval-S per-type accuracy, full 500 questions (Qwen3-32B served at 128k via YaRN; internal Qwen3-32B judge; 95% bootstrap CIs on *answerable* in §4.3). Two regimes: when the 115k haystack *fits* the window, full context wins overall (answerable 0.564), but routed selection at 4.5× less budget *leads* on temporal and knowledge-update; at a 28k budget where the history must truncate, routed selection (0.496) beats truncation (0.302) by +19 pp, and the embedding-hybrid router beats lexical by +4 pp. Correct-abstention (not shown) is 0.60/0.67/0.67/0.77.

4.4 Where the gain comes from: payload fidelity, not the router

It would be tempting to credit a clever router. A same-batch memory×reader factorial (Table 6) says otherwise. The verbatim-selection substrate accounts for most of the memory-side gain—*just selecting raw spans*, with a token-overlap router and the harness's default reader, already matches the purpose-built system (0.5601 vs 0.5722), and the full memory stack under that same default reader beats it (0.5954, +2.3 pp). Routing refinements are second-order: the fused second router signal adds +1.4 pp as originally measured (audited and superseded in Appendix F) and the structural step-pin +1.7 pp, consistently at either reader (+3.1–3.5 pp total). The single largest effect is not in the memory at all: swapping the reader's one-sentence answer instruction for a structured one adds +5.1–5.5 pp at either router, and the two axes are additive to within batch noise (same-day replicate ±0.5 pp). Memory systems ship with very different readers (AMA-Agent an agentic tool loop, MemGPT its own control loop), so end-to-end gaps that do not control the reader conflate the two effects; we report them separately. Router *sophistication* remains regime-dependent: its payoff scales with how lexically un-addressable the evidence is—≈ 0 on step-indexed agent trajectories (audited, Appendix F), rising to +4.3 pp on long chat (LongMemEval-S) and +12.6 pp on diffuse dialogue (LOCOMO), where it is the only thing that lifts selection above full context. Below we localize this memory-side win to the *fidelity* of the payload, show that serving that payload as cached KV rather than re-encoded text is faithful in a different, *computational* sense—one that carries a cost advantage but, we find, no accuracy advantage—and treat the reader as a separable axis whose answering protocol can be served from the memory itself.

Substrate, then structure, then semantics. On AMA-Bench specifically, three memory-side effects separate cleanly (tabulated side by side in Appendix G). The *substrate*—serving selected raw spans verbatim in a filled budget, even with a free token-overlap router—is the bulk of the gain, lifting us from the constructed-memory field into AMA-Agent territory; selecting raw spans alone is not, since AMA-Bench's own raw-turn baselines score 0.34–0.42 (*Versus standard vector-RAG*,

Table 6: Same-batch memory×reader factorial on AMA-Bench (brackets: 95% episode-clustered bootstrap CIs, 208 clusters). The two axes are additive: routing adds +3.1–3.5 pp at either reader, and the structured answer instruction adds +5.1–5.5 pp at either router. Under the *default* reader the memory stack alone already scores above the purpose-built AMA-Agent (0.5722). The lexical/default cell replicates Table 3's lexical row (0.5601 vs 0.5621, different days); the deployed/structured cell is a local run of the configuration the leaderboard verifies at 0.6478.

Memory configuration	Default answer prompt	Structured answer prompt
Lexical router	0.5601 [0.535, 0.585]	0.6154 [0.591, 0.640]
+ deployed router + step-pin	0.5954 [0.570, 0.620]	**0.6466** [0.622, 0.672]

below). Semantic router *sophistication* is second-order at best (audited below). The remaining lever is not a smarter scorer but a one-line *structural* cue: 73% of AMA-Bench questions cite a step index ("at Turn 38", "across steps 28–30"), and pinning exactly those steps adds +1.7 pp—and does so *entirely* on the step-citing questions (+2.5 pp on them vs exactly +0.0 pp on the rest), a clean mechanism signature: on step-indexed agent traces the order is substrate ≫ semantic router ≈ structure, and the structural pin is a transparent rule, not a learned exploit.

The deployed router, audited. The router in the deployed stack fuses lexical overlap with a second "model-pick" signal. A controlled full-set audit finds that signal *score-neutral*: paired within one batch, a lexical base + step-pin (0.6466) differs from the model-pick hybrid base by −0.8 pp [−2.3, +0.8]; with its pick call fixed, the signal stays score-neutral.[8] We therefore name and release the reproducible **lexical + step-pin** default—the substrate and the pin carry the routing on AMA-Bench, not a semantic scorer.

Robustness of the step-pin result. The step-pin is a transparent bonus on the step-citing subset, not the source of the result: deleting the step number from the query costs content-based selection only −0.2 pp, corrupting the pin address degrades gracefully rather than catastrophically, and a different-family Llama-3.1-8B judge preserves every ordering.[9]

Controlled payload ablation (oracle evidence). The cross-system +10–35 pp above conflates payload form with chunking, prompt, and budget. We isolate the payload with a within-system test: on step-citing questions we take the *exact cited step* as oracle evidence (holding retrieval fixed) and present that same evidence three ways—the raw *verbatim* step, an LLM *summary*, or LLM-extracted *facts*—to the same reader, prompt, and judge. Only the payload form varies, so the gap is causal (Fig. 3): verbatim beats an LLM summary by +10.1 pp and extracted facts by +8.5 pp over 910 QA, widening to +14 pp on exact-recall (type A) where lossy forms drop the exact value. This is the controlled version of that result: matched evidence, matched reader, lossy re-encoding *alone* costs 8–14 pp. To rule out the reader grading its own generations leniently, a *different-family* judge (Llama-3.1-8B) re-scoring the same answers preserves the ordering—verbatim 34.3% > facts 30.2% > summary 29.8%, still +4–5 pp—so the verbatim advantage is not a judge artifact. (These absolute numbers are not comparable to Table 3: this stripped setting gives the reader only the single cited step—no overview, recent turns, or neighbors.) A *same-pipeline* version closes that gap in scale: keeping the full deployed context and replacing only the router-selected verbatim appendix with LLM summaries of the same steps (same QA, overview, router, reader, and judge; the swap fires on 96% of questions) costs −5.1 pp (0.578 vs 0.629, same-day). Table 7 completes the family under same-batch anchors: swapping the appendix to Mem0-style *extracted atomic facts* costs −5.9 pp, and both generative re-encodings score at or *below* serving no appendix at all (gist-only −4.3 pp)—a lossy re-encoding of the selected evidence adds nothing over the one-line gists already in the overview. The control that separates *compression* from *paraphrase* is a deterministic swap to the query-relevant *original lines* of the same steps (no LLM, no rewording): it costs only −2.6 pp, about half the generative cost in the same pipeline. So lossy *generative* re-encoding is far more harmful than extractive compression—which still costs −2.6 pp—and in this pipeline the choice is verbatim (whole spans or extracted original lines), not construction.

[8] Full audit in Appendix F.

[9] Appendix G.

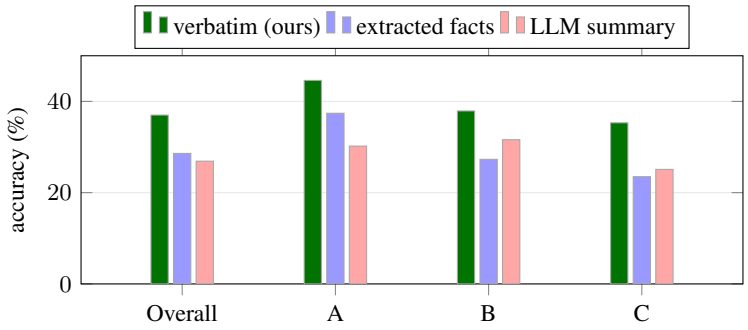


Figure 3: Controlled payload ablation (matched oracle evidence, reader, prompt, and judge; 910 step-citing AMA-Bench QA; type A = exact-recall)—only the payload *form* varies. Re-injecting the selected steps *verbatim* beats lossy re-encodings on every question type: $+10.1$/$+8.5$ pp overall over LLM-summary/extracted-facts, widening to $+14$ pp on exact-recall. Lossy re-encoding of matched evidence *alone* costs 8–14 pp—isolating the substrate.

Table 7: Same-pipeline payload attribution (1,248-QA half): each arm differs from the deployed system *only* in what fills the evidence appendix; the first row instead replaces the assembled context. Δ is against each arm's same-batch structured anchor.

Context / appendix content	Acc	Δ (same-batch)
Full trajectory, recency-truncated (no memory)	0.5577	-8.4 pp
Appendix $\rightarrow$ LLM summaries (same steps)	0.5777	-5.1 pp
Appendix $\rightarrow$ extracted atomic facts	0.5825	-5.9 pp
No appendix (recency+gist overview only)	0.5865	-4.3 pp
Appendix $\rightarrow$ query-relevant *original lines* (no LLM)	0.6154	-2.6 pp
Appendix = selected steps verbatim (deployed)	0.6290 / 0.6418	—

The non-selected floor is second-order; payload fidelity is the lever. Routed selection and the gist overview are two *separate* assembly choices, so we cross them in a same-batch 2×2 (Table 12, Appendix D). At the deployment budget the four cells are statistically indistinguishable—every paired contrast is within 0.9 pp with McNemar $p > 0.28$. The gist overview is a navigation scaffold, not a payload: adding it to plain recency changes accuracy by -0.4 pp (n.s.), with or without selection. The routed verbatim appendix adds little *at this lexical router*—it has not yet found the right spans—but its value scales with selection quality, reaching -4.3 pp to remove at the deployed router+pin (Table 7). The variable that moves accuracy is therefore neither the overview nor the bare presence of an appendix, but the *fidelity* of what selection returns: the same routed spans served verbatim rather than as an LLM summary are worth $+10.1$ pp (Fig. 3). Representation of the floor is second-order; fidelity of the selected payload is first-order. This selection-versus-overview split is domain-dependent—selection helps retrieval-heavy Web, SOFTWARE, and TEXT2SQL and slightly hurts holistic Game and OpenWorld—which we break out in Appendix D.

Versus standard vector-RAG. The embedding router *is* a standard vector-RAG retriever—it embeds each step's full text and takes cosine top-k, the same first stage as AMA-Agent, whose released code likewise re-injects the retrieved raw steps (next to an LLM-written state summary). Verbatim re-injection is therefore not, by itself, what separates VeraKV: AMA-Bench's own raw-turn baselines also return raw steps (BM25, top-10: 0.3436; Qwen3-Embedding, top-5: 0.4227 (Zhao et al., 2026)). On a shared subset our routers order lexical 0.557 $<$ vector-RAG 0.583 $<$ RRF hybrid 0.600—the router moves accuracy by about 4 pp—while all three sit far above those raw-turn baselines and every constructed-memory system (Table 3). That gap is cross-pipeline and mixes budget, assembly, and prompt, so we do not attribute it to verbatim re-injection alone; VeraKV adds tiering, structural pinning, and KV reuse on top of a standard retriever.

Table 8: Serving the selected spans against a full-prefill-then-mask oracle (§4.7). Position-preserving KV gather reproduces the oracle; re-encoding the same span as isolated text does not, because it loses the trajectory-conditioned context the cache retains.

Serving the selected spans	Δlogit to oracle	1st-tok argmax	256-tok seq
Gathered KV at orig. positions (ours)	~ 0.7 (bf16 noise)	100%	identical
Compact re-prefill as isolated text	~ 13.5	—	diverges

4.5 SERVING THE EVIDENCE AS KV: THE MODEL'S OWN READING

The payload-fidelity result has a limit: text re-encoding, however verbatim, is still a *fresh* encoding of the evidence. The faithful ideal is the model's *own* reading of it—the keys and values it already computed over the trajectory—and because VeraKV's store is a KV cache, we can serve exactly that. Selection decides *which* evidence reaches the reader; the KV cache decides *how faithfully* it is carried, and these are the two halves of VeraKV. Because the store is a KV cache, selecting is a cache operation: we prefill the trajectory once and, per query, gather the selected spans' keys and values *at their original positions* and decode only the question. Each span's KV was computed *in situ*, while the model attended to everything before it, so it holds that span's trajectory-conditioned reading. Position-preserving gather is therefore exact under the conditions it relies on—identical cached keys and values, retained *original* position ids, and a standard causal RoPE transformer (mechanisms that read absolute position, sliding-window or sink masks, state-space mixing, cache quantization, or paged relocation fall outside the guarantee)—so the sub-selected prefill reproduces a *full-prefill-then-mask* oracle: first-token argmax matches on every query (100% across synthetic and 144 real AMA trajectories, $24/24$ episodes), and greedily decoding 256 tokens it emits the identical sequence as the oracle at both 3k- and 27k-token trajectories (logit-KL ≤ 0.011; §4.7).

The exactness survives to graded answers. The oracle match is at the logit level; it also holds end to end. On the 112 real AMA episodes that fit the prefill window ($\leq$ 26k tokens—five of the six domains, since SOFTWARE's 60–100k-token trajectories cannot be prefilled whole, cf. Table 1), we route the *same* spans per query and answer all 896 questions *both* ways—gathering the selected KV at its original positions versus re-prefilling those spans as text—then grade both with the same Qwen3-8B judge. Accuracy is at parity: 17.3% from the cache versus 17.4% from text ($\Delta\, 0.1$ pp, one question in 896; each question type within ± 2 pp), with 83% first-token argmax agreement and the mask-oracle faithfulness gate holding throughout ($126/128$). The absolute level reflects a stripped 8B harness (k=5 selected spans, no overview), not the headline setting; what the test isolates is that the two serving paths are *indistinguishable downstream*. Serving the model's own cached reading is therefore not merely logit-equivalent to re-encoding the evidence, it is *accuracy*-equivalent wherever the trajectory fits the window.

The cache carries context that re-encoded text cannot. This is where a KV memory departs from a verbatim-*text* one. Re-encoding the same selected span as text *in isolation*—the compact pipeline—starts fresh: its tokens never attended the dropped surroundings, so its keys and values are not the ones the full model computed. Measured against the mask oracle, gathered KV sits ~ 0.7 logits away (bf16 non-determinism), while a compact text re-prefill of the same span sits ~ 13.5 logits away—about $20\times$ farther (Table 8). The gathered KV therefore carries trajectory-conditioned context that no re-encoding of the span in isolation can reconstruct. A direct causal test confirms the cache carries this and text cannot: perturbing only the *unselected* upstream turns and re-gathering the *same* selected spans changes the gathered-KV answer on 66% of queries ($111/168$ real AMA QA, Qwen3-8B), while a text re-prefill of those spans—blind to the perturbed turns—is invariant *by construction* (0%), the gather staying faithful to the mask oracle throughout ($24/24$). This is the deeper reading of "deliver evidence faithfully": the cache is the model's own reading of the evidence, served unaltered—and it is what the name VeraKV refers to. Everything above holds within the window, where every served row descends from one joint prefill; §5 charts what survives beyond that boundary, where no joint prefill exists and exactness is impossible in principle.

4.6 THE READER IS A SEPARABLE AXIS

Reader comprehension is a separate axis that memory research usually leaves entangled with the memory—systems ship with very different readers, so a gain credited to a memory can be a reader change instead. Holding the reader fixed, the two axes prove asymmetric: a memory error is unrecoverable downstream, while every reader-side deficit we found is repairable at answer time by one sentence. Stress-testing fourteen reader-side mechanisms over a fixed memory—plans, compiled views, checklists, re-retrieval, type-specialized instructions—all come back null or negative once routing reaches $\sim 100\%$ recall; the single lever that reaches emission is the one-sentence *answer instruction* ($+5$ pp), and even that is answer-completeness, not extra reasoning.[10]

That one lever is itself *serveable*: prepending the structured answering protocol to the front of the retrieved context—a fixed-position segment prefilled once under KV serving, leaving the host instruction untouched—transfers most of the gain ($+2.3$ to $+3.8$ pp over same-batch anchors). In the out-of-window store regime (§5) the transfer becomes exact: the instruction encoded *once* as a global skill block (no episode content) and spliced per query with zero forward passes—its keys RoPE-rotated to sit contiguously before the question—ties the text instruction on both the anchored-store and the harvested-cache bases (-0.4 pp, p=0.86; $+0.1$ pp, p=1.0; full $+4.5$/$+8.1$ pp lift transmitted; the parity replicates on LOCOMO at 8B and 32B, §5), while the same block frozen at a far position transmits only a fraction of it. The transplantation mechanism itself—precompiled KV made position-portable by RoPE repositioning—is established concurrently by Li (2026) across twelve models; what we add is the memory-side characterization: the protocol travels as a retrievable object of the same store that serves the evidence, at answer-accuracy parity on official-scale memory QA. The deployed system holds a *single* fixed protocol block per benchmark. Whether a query-routed *library* of blocks beats that single block is a question the same splice answers directly, and we now run it: eleven skill blocks, regex-routed per question with the universal structured answer protocol as fallback, served as rotated KV. It resolves the routing question our falsification map had left open. The *replacement* form still fails—swapping the universal instruction for type-specialized fragments regresses -3.0 pp with ten collateral losses per targeted fix (App. C). But the *additive* form wins: a per-class hint appended to the intact protocol lifts accuracy $+2.1$ pp overall (p=0.009, n=800) and $+6.3$ pp on the 34% of questions where a rule fires (up to $+10.8$ pp on the largest routed class, n=130; per-class figures are descriptive, from a single fixed rule set rather than a swept design), whereas replacement over the same rules stays a wash ($+3.7$ pp routed, p=0.27) and double-edged ($+12.5$ pp on one class, -15 pp on another). Served as KV the blocks show no detectable gap from their text form across all three variants ($|d| \leq 1.8$ pp, all $p \geq 0.25$)—an absence of a detected difference, not a formal equivalence—so the library precompiles into the same store, though the KV-carried additive gain is only directional at this scale ($+0.8$ pp, p=0.46), leaving the text carrier the demonstrated lever. The store's answering skill is therefore not a fixed constant but a routed, extensible object of the memory: additive routing a real headroom lever, replacement a trap. Placement, not context, is what a skill needs: it never attended the evidence and works at full strength—the mirror image of evidence blocks, which demand native positions and context-conditioned encoding.

4.7 EFFICIENCY: KV SUB-SELECTION DECOUPLES COST FROM EVIDENCE

Serving spans from the cache rather than re-encoding them is correct by construction—position-preserving gather reproduces the full-prefill computation, an equivalence that holds across attention kernels, a $4\times$ YaRN extension, and a non-Qwen backbone.[11] We measure its *cost* as a single-request microbenchmark. Because the sub-selected decode re-prefills only the question, its time-to-first-token is *flat* while the text path grows with the selected evidence (Fig. 4): on a dense Qwen3-8B the per-query speedup rises from $1.6\times$ to $10.9\times$ as evidence grows to $\sim$4k tokens, reaching $32\times$ at 32k, and the one-time trajectory prefill saves $\sim$2M prefill tokens over a full AMA run. The decoupling is a property of the gather, not the weights—a Llama-3.1-8B backbone shows the same flat TTFT.

Two fairness notes on this comparison. First, the speedups are measured against re-prefilling the selected evidence as text; the strongest within-window alternative is a *resident full-trajectory prefix cache* (standard prefix caching), which also re-prefills only the question—against it, sub-selection's

[10] Full 14-mechanism falsification map in Appendix C.

[11] Correctness gauntlet and end-to-end accuracy-neutrality against the text pipeline in Appendix H.

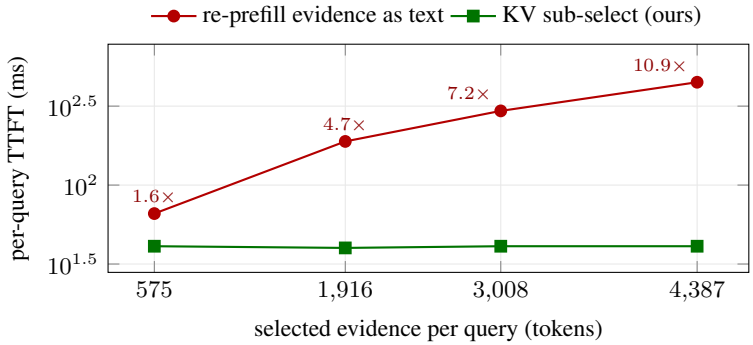


Figure 4: KV sub-selection decouples per-query latency from evidence size. As a query selects more evidence, re-prefilling it as text grows TTFT (66 → 448 ms; dense Qwen3-8B), whereas gathering the selected spans' KV re-prefills only the question and stays flat (∼ 40 ms); the widening gap is the per-query speedup (labeled, 1.6× → 10.9×), reaching 32× at 32k selected tokens (§4.7). First-token argmax matches the full-prefill oracle at every point.

saving is not the prefill but the attention context (every question and decode token attends E selected tokens rather than all N), plus the option to evict unselected rows; measured on the same protocol ($N{\approx}14$k resident rows, $K{=}2$–12), the sub-selected question reaches first token in 40–48 ms versus 79 ms over the resident full cache (1.4–1.9×) and 140–369 ms for text re-prefill, decodes 26% faster per token (32.7 vs. 41.2 ms/token), and the gather itself costs 3–6 ms. Second, the saving is in the *prefill*, and it is bought with memory: the resident KV grows at 0.144 MB/token (9.4 GB for a 64k-token episode). A like-for-like comparison against production paged-attention stacks needs a serving-system integration, which we leave to future work.

5 Out of the Window: When Can Agent Memory Reuse KV Caches?

Within the window the cache path is exact because every served row descends from one joint prefill (§4.5). Beyond it, exactness is impossible in principle: once the history outgrows the window, events must be encoded without their future and often without most of their past, and any such store is a different computation from the full reading. This section charts what survives the crossing. Throughout, arms are paired per question (exact McNemar tests, episode-clustered bootstrap CIs), all arms see the same routed evidence unless the contrast itself is evidence visibility, and the pipeline is deterministic—the deployed configuration reproduced byte-identically nineteen times across independently written harnesses.

Stores, and the repair. Each event (agent step span) is encoded offline into a per-event KV store and a query selects K of them (§2); the stored rows are assembled at their *original* positions behind the shared header and the question is decoded over them. The text path (*tx*) re-prefills the same routed evidence as text. The naive store (*iso*) encodes each event in isolation. The repaired store (*anchored+tail*) instantiates two known repair families as agent-memory write/serve operations—we adapt and characterize here; the primitives and their combination are prior art. At write time, *anchored encoding* (Acharya et al., 2024): each event is encoded as [header; episode opening ($w{=}4096$ tokens, truncated at the event start, at its native positions); event], keeping only the event rows. At query time, *selective recomputation* (Yao et al., 2024; Hu et al., 2024): the opening's rows not covered by selected events are served alongside, and the last two events are recomputed freshly over the assembled cache before the question.

Failure, repair, and the budget. Table 9 is the headline. Isolated stores lose 13.2 pp against text re-prefill of the same evidence—to our knowledge the first large-sample *paired* quantification on the official harness, though the phenomenon itself is established (MemTensor, 2026), and our failing object is construction-time independent encoding, not transplanted prefixes. The repaired store recovers 72% at $K{=}12$; at the matched lean budget $K{=}5$ the paired difference is statistically indistinguishable from zero (-0.8 pp, CI $[-3.0, +1.4]$)—we do not call this non-inferiority, since

Table 9: Out-of-window portability on the official AMA-Bench harness (Qwen3-32B reader, official judge, paired; n=2431 at K=12, 2403 at K=5). Isolated event stores collapse; the repaired store recovers 72%, and at the lean budget the paired difference is statistically indistinguishable from zero, with a CI that excludes regressions approaching the naive failure (the margin was not pre-registered). tx here is the routed-text *carrier* baseline under the serving-cell protocol (fixed K retrieval, no overview; Appendix A)—*not* the full deployed pipeline of Table 3; any within-window cache-vs-text edge elsewhere is evidence-matched or re-attributed per the matched-evidence audit.

budget	serving	score	paired vs. tx
K=12	re-prefill routed text (tx)	.397	—
	isolated stores (iso)	.265	-13.2 pp, p=10^{-38}
	anchored+tail	.360	-3.7 pp (p=10^{-4}): recovers 72% of the drop
K=5	re-prefill routed text (tx)	.368	—
	anchored+tail	.360	-0.8 pp, clustered 95% CI $[-3.0, +1.4]$ pp

the margin such a claim needs was not pre-registered; what the CI does exclude is anything resembling the naive failure. Beyond the window, where the text path must re-prefill everything it serves, the cache path reaches parity at 12–18% of the text path's per-query fresh tokens. Costs are stated honestly: writing stores costs $\sim$5$\times$ one episode prefill (offline, event-parallel), repaid over-window and under reuse, not within one episode's own questions; in-window TTFT is 1.4–2.3$\times$ faster but decode is $\sim$29% slower under the scattered layout, so end-to-end latency inverts for answers longer than $\sim$20 tokens—the claim is *reduces TTFT*, never *answers faster*.

Matched evidence, and the ceiling. Two audits discipline the wins. First, the cache path's within-window edge over routed text (+5.6 pp at 8B) is *not* a carrier effect: served anchor rows are extra evidence, and giving the text path the same rows (tx_{plus}) recovers most of the gap (+4.3 pp, p=0.001 at 8B; +5.5 pp, p=0.004 at 32B); at matched evidence the two carriers are at parity at both scales. The episode opening is simply cheap evidence that K-routing systematically misses—the cache gets it structurally, text must buy it with tokens. Second, against *complete* reading: at 8B the repaired store *ties* a full re-prefill of the entire episode (.221 vs. .220, p=1.0) at $\sim$22% of its query cost—while rows *gathered* from that same full prefill score far below it (.175): exact rows minus answer-time global attention lose what approximate-but-repaired stores reconstruct. Gathering itself is not the culprit: give the gathered rows the same opening-region rows—all free from the same cache—and the harvest path reaches statistical parity with the repaired store (-1.1 pp, p=0.37), *with no fresh tail at all* (the tail, worth +3.9 pp on isolated stores, buys exactly nothing on harvested rows, which already carry full-context integration). Where a rollout cache exists, then, a *zero-write* memory—gather routed spans plus the opening from the agent's own run cache—matches the 5$\times$-write recipe: exactly so at 32B (.3313=.3313, $+57/-57$ discordants, both the identical 5.3 pp below the strong reader's full-reading ceiling), and robustly to prompt mismatch (harvesting from a cache built under an agent-style system prompt costs +0.1 pp, n.s.). The pattern also crosses benchmark families and scales: on LOCOMO (n=1150 paired, sessions as events) the zero-write harvest exactly ties the text path at 8B (+0.1 pp, p=1.0) and sits at-or-above the repaired store (+2.0 pp, clustered CI $[-0.1, +4.2]$), spending 40 question-only fresh tokens per query against the text path's $\sim$9k; at 32B, where the repaired store's published edge over text replicates to the decimal (+3.9 pp, CI $[+1.9, +6.0]$, p=8×10^{-4}), the harvest inherits it (+3.4 pp over text, p=0.004; -0.5 pp vs. the store, p=0.64), and the answer instruction carried as a rotated skill block matches its text-carried control at both scales (+0.1 pp p=1.0; +1.0 pp p=0.24). It also crosses backbones: on Llama-3.1-8B-Instruct the full cell replicates—unrepaired global holes fall below the store (-2.2 pp), the opening-repaired harvest sits at-or-above it (+1.8 pp, CI $[-0.2, +3.9]$), and the fresh tail again buys nothing on harvested rows (+0.2 pp/-0.5 pp, n.s.). What the write buys is the over-window regime, where a live run truncates and no full-context cache ever exists. The tie is reader-scoped: a 32B reader keeps converting complete context into accuracy (+9.1 pp over routed text, p=10^{-5}), leaving every budget-limited serving—cache and text alike—below it and at parity with each other. The residual is not degeneracy: in the official losses the cache path's answers are 98% fluent with wrong precise details (zero garbles; degenerate outputs are equally rare in both paths), and distributionally the cached serving sits *closer* to the full-reading distribution than routed text does (median first-token KL 0.005 vs. 0.031 nats)—losses are near-tied argmax flips, which is also why no cheap per-question

fallback trigger exists (six signal families, including running both paths and voting, all fail to beat chance).

Regularity I: evidence converts to accuracy only through joint encoding. (Both regularities are empirical and scoped: one model family, this benchmark family, these budgets.) Raising the routing budget $K{=}5{\to}12$ moves the two carriers oppositely: text gains $+2.8$ pp ($p{=}0.0015$) while the cache is *exactly* flat (.3593=.3593)—not insensitive but self-cancelling ($+199/-199$ flipped answers). A within-episode factorial sharpens this into the paper's cleanest controlled statement: handing the *same* three extra events to both paths, text gains $+2.6$ pp while the cache *loses* -1.8 pp (difference-in-differences $+4.4$ pp, clustered CI $[+1.5, +7.3]$), and for the cache *any* extension hurts (random: -1.65 pp, $p{=}0.029$). Freshly re-read text is jointly re-encoded with the question and the rest of the evidence; pre-encoded blocks that never attended each other add retrievable-but-isolated facts plus attention dilution, and the two cancel. Deployment follows: serve the cache *lean* ($K{=}5$ serves $\sim$40% of the $K{=}12$ volume for free), and to close the residual add fresh computation or reader scale, not served rows.

Regularity II: write-time conditioning must match what is served—in content and in position. Why does the anchor work? Not by injecting content: with serving matched and equal-length conditioners, a true-episode prefix beats no conditioner by $+3.3$ pp ($p{=}0.009$), but another episode's text or even a *shuffled* prefix performs similarly, while degenerate fillers (a repeated token, a generated digest) score *below* no conditioner—the anchor primarily repairs the encoding boundary state (sink placement, positional posture), and a wrong conditioner is worse than none. A *note-transplant* reading of this repair—prefill parks derived conclusions on delimiter/aggregator tokens, and serving those rows would deliver them (Li, 2026)—fails a direct test: serving only the anchor region's delimiter rows does no better than matched random rows ($-0.7/+0.4$ pp at $K{=}64/256$, both n.s.), both trail serving *nothing* at $K{=}64$, and the region's $+4.5$ pp value ($p{=}3{\times}10^{-4}$) appears only as the near-complete contiguous block—a scaffold-and-coverage effect, not sparse conclusion notes; sparse rows spliced into an assembled cache make holes, they do not carry conclusions. Two controlled dials then bound the remaining semantic and geometric components. *Content*: freezing one global conditioner for all episodes—which would make writes prefix-cacheable, collapsing write cost $5\times \to 1\times$—costs -3.4 pp ($p{=}0.007$): the same-episode content contributes a real 2–3.4 pp. *Position*: sliding the anchor adjacent to the event (the contiguous layout every splice-and-repair pipeline assumes) costs -2.8 pp ($p{=}0.0008$), while pushing the event 8k tokens *farther* away costs almost nothing (-1.7 pp, n.s.), and distance mutes neither the benefit nor the poison of the conditioner. The store works best when its write-time worldview matches the serve-time layout—the opening at its true position, the hole between them real, the served anchor rows the ones the store was conditioned on. Query-time token repair cannot substitute for this write-time conditioning: an EPIC-style static boundary recompute (Hu et al., 2024) at its published budget is a null in this regime ($k{=}16$: $+0.2$ pp, n.s.), and even equipped with our served anchor rows and fresh tail it trails the anchored store by 5.2 pp ($p{<}10^{-4}$)—in agent traces the isolation damage is distributed over the whole event's rows, not concentrated at its boundary. Nor does a *smarter* conditioner help: replacing the opening with each event's top dependency-graph upstream events (budget-matched, native positions, their rows served) loses -2.9 pp ($p{=}0.022$) and is no better than random prior events—while serving the conditioner's own rows helps for *every* conditioner family ($+2.8$ to $+3.4$ pp), generalizing the consistency law. The opening wins because, alone among the six write-side alternatives we tested, it is simultaneously natural text (boundary repair), same-episode content, truthful geometry, the evidence block routing systematically misses, and a constant fully-serveable block; the six write-side alternatives we tested all lose to it.

The fresh budget: six selectors tested, recency stands. The one query-time degree of freedom is where the two-event fresh recomputation goes. At fixed evidence the tail helps immediately and saturates ($+2.1$ pp at two events, flat at six), and its value is not query relevance: under matched evidence and budget, six selectors—recency, a signature-co-occurrence dependency graph, query$\to$cache attention mass, a KEEP-style query-conditioned attention propagation over events (the group-recompute selector of Yang et al. (2026), instantiated at our event granularity), query relevance, random—leave recency tied or ahead. The propagation selector lands exactly on the random floor ($+0.2$ pp vs. random, $p{=}0.89$) and significantly below recency (-2.1 pp, clustered CI $[-3.5, -0.7]$, $p{=}0.016$), despite spontaneously agreeing with recency on 1.2 of its 2 picks; query-

selected excerpts are marginally *worse* (third replication of that null). The selectors do pick different events; the differences do not pay. Recency is not a lazy default but the causally downstream integration point, and the operating point is thereby six-way validated rather than assumed.

Deployment map. The recipe splits by *where the rows come from*, not by what is served. In-window, where the agent's own rollout cache survives, harvest it: no store encoding, no fresh tail, robust to the run-vs-review prompt mismatch, at exact parity with the re-encoding recipe at both scales (§above). Over-window, a live run truncates and no full-context cache ever exists—there, and only there, the $5\times$ write buys anchored stores that are window-independent. In both regimes the reader's answering protocol travels as a rotated skill block from a global store (§4.6), and the two memory content types obey mirrored serving rules: evidence rows are position-preserving and context-hungry; skill rows are position-independent, query-adjacent, and context-free. Route by domain, not per question: on the official judge the cache path wins EMBODIED outright at both budgets (.19–.21 vs. .12–.13) and text keeps OPENWORLD; with domain routing the indistinguishable-at-$K{=}5$ headline becomes a win where traces are dense state–action loops. Reader capacity moderates everything: on LOCOMO the repaired cache beats text by $+3.9\,\mathrm{pp}$ at 32B (conversation-clustered CI $[+1.9, +6.0]$) while 8B regresses on single-hop near-details; 77% of those regressions are answered correctly at 32B under the same protocol and memory content. The end-to-end deployment-map system—harvest+skill in-window, anchored stores+skill over-window—was then run once on the full official harness against the pre-registered paired contrasts: over $n{=}2431$ paired questions it scores .397 against the anchored-store recipe's .360 ($+3.6\,\mathrm{pp}$, clustered CI $[+1.7, +5.6]$, $+304/-216$ discordants, $p{=}1.3{\times}10^{-4}$) and *exactly ties* the routed-text carrier baseline of Table 9 (.397 vs. .397, $+285/-287$ discordants, $p{=}0.97$)—the serving-cell gap closes; the full deployed pipeline (different protocol, Table 3) is not the comparator here. The gain lives on the in-window slice (43% of questions, $+6.4\,\mathrm{pp}$, $p{=}9{\times}10^{-5}$), where the zero-write harvest serves the rows and the skill block delivers the answering protocol; there the KV-carried skill also matches its text-carried control ($+1.6\,\mathrm{pp}$, n.s.), and the over-window slice is unchanged ($+1.5\,\mathrm{pp}$, n.s.).

Four synthetic probes of what a served row carries. The regularities above are attribution claims about a served row—what computation it still embodies once detached from its origin. Four controlled probes make the mechanism direct (Qwen3-8B, synthetic register-tracking traces after RULER variable tracking (Hsieh et al., 2024), exact-match scored, arms paired), spanning the four ways a row is produced: one continuous prefill (*full*), one joint prefill then *gather* the wanted rows (*harvest*), independent per-event encoding $[\text{header}; E_i]$ (*iso*), and a rolling anchored write $[\text{header}; \text{opening}; E_i]$ (*anchor*). **(1) Phantom conclusions.** Two equal-length prefixes carry opposite states; the *same* event text ("the status from step 1 was copied into the report") is teacher-forced after each, and we serve *only* that event's rows—identical token IDs, positions, header, and row count across the two arms, so only the invisible donor differs. Across 72 such donor pairs, both arms separate in their donor's polarity direction in 64% of pairs (each arm alone follows its own donor 81% of the time) against an 8% no-donor floor, while the donor's *exact* token is reproduced only 10% of the time (significance taken over the 72 pairs, not the 144 arms)—a served row is a lossy carrier of the *conclusions* its prefix induced, not merely a cache of its own visible tokens (attention-mediated memoized inference (Li, 2026), measured). **(2) Erasure boundary.** That carry is not *unconditional* leakage. Deleting a numeric value's source rows from a joint prefill and then querying the current state, the model answers "not specified" exactly as an oracle that never saw the value ($0/96$; an overwriting event cleanly supersedes, $96/96$): no *detectable* behavioral leakage under explicit overwrite and current-state queries. We claim only that—$0/96$ bounds the leakage rate below $\sim 4\%$ and says nothing about logit-level residue, arbitrary queries, or exact state-equivalence to a fresh prefill (§7). Read with (1), the picture is selective: a prefix's influence surfaces when the model formed a task-relevant conclusion in the suffix *and* the query reads it, not whenever the prefix was present. **(3) Sink versus dependency.** On a three-event register chain a shared header lifts multi-hop resolution from 0.20 to 0.48 ($p{=}2{\times}10^{-11}$) but leaves it short of full context's 0.72 ($p{=}6{\times}10^{-8}$); single-fact retrieval shows no improvement (0.18 vs. 0.14, both far under full's 1.00—a retrieval floor under independent encoding, not a null interaction). A shared prefix therefore substantially but only partially restores cross-event reasoning over independently encoded rows; it does not *store* the cross-event edge, the residual to full being the cost of the missing joint encoding, which is why sharing a prefix (Star-style (Acharya et al., 2024)) addresses repeated-sink placement yet not the missing inter-event conditioning. What in the header carries this—sink normalization and positional posture versus the variable and instruction semantics it also supplies—we do not

separate here; an equal-length sink-only header would, and we leave that open. **(4) Where the harvest gain comes from.** A four-arm decomposition on the same trajectory-spanning chains settles the attribution (max sequence 589 tokens, so full's failure is distraction, not truncation; exact- and number-recall scoring agree to the digit). Serving only the gold chain reaches 0.98–1.00 however it is carried—fresh compact text (1.00), independently encoded rows (1.00), or rows harvested from one joint prefill of the full trajectory (0.98)—while full context sits at 0.04. The whole gap is *routing/denoising*: selected-text − full = +0.96 (+368/−0, $p<10^{-100}$). *Pre-digestion* is null—harvest does not beat a clean re-encode of the same chain (−1.8 pp, if anything slightly worse for having attended the distractors)—and *joint conditioning* is null on the routed chain—harvest ties independent encoding (−1.8 pp), because once only the relevant events are co-served the reader re-derives the chain at serve time regardless of carrier. So under *complete* routing harvest buys *cost*, not computation: a near-lossless carrier of the selected rows (within 2 pp of fresh text), the accuracy lever being distraction removal, which any selective serving delivers. Its globally computed rows out-earn a fresh re-encode only in one narrow corner—when routing is *incomplete* and the missing content is a coarse, explicitly copied label. Dropping the step that introduces the target and serving only a later row that copied it: a binary state survives almost perfectly—but our first two measurements of that cell were both wrong in instructive ways, and the corrected chain is worth stating. The original design drew the served decoy as the gold label's negation; at $K=2$, "recall the memoized value" and "answer the opposite of the served decoy" are indistinguishable, and the arm read harvest 0.99 against 0.00 for independent encoding. Redrawing the decoy *independently* ($n=256$) exposes fresh text and independent KV as pure decoy echoers—both score ≈ 0.5, exactly the rate at which an independent decoy happens to coincide with the gold—and puts elimination-free recall at $r \approx 0.80$. That 0.80 was in turn a *readout* limit, not a storage limit: naming the two candidate labels in the question lifts recall to 0.934–0.984 across three runs ($n=256$ each), the cleanest being a cell with *no* state word served anywhere—nothing to echo, nothing to eliminate—where harvest scores 0.984 against 0.54 for both re-read text and independent KV (coin flips, as they carry nothing). Entropy still closes the channel, and the menu defends the boundary rather than dissolving it: with candidates named and decoys independent, a four-way label sits at chance (0.223 vs. 0.25; 92% of answers echo the served decoy), an eight-way label likewise (0.156 vs. 0.125), and an exact three-digit value never survives (0/128). A verdict that must be *computed* rather than copied is not pre-stored either (0.48, at chance under the same menu-style readout—though the model derives it fine when the source *is* served). The failure mode is uniform: when the row cannot supply the answer, the reader echoes whatever label is served, which is also why harvest is not uniformly better—on the subset where the independent decoy coincides with the gold, the echoers score a lucky 1.00 while harvest's residual recall failures cost it (0.803, $p=3\times10^{-8}$). What a row inherits also turns out to be sharply *local to its own predicate*, not a side channel for whatever preceded it: with a verdict explicitly stated upstream and then dropped, a semantically neutral later row ("checkpoint saved for the sensor review") carries nothing (0.47, chance), and neither does a row *about* the verdict ("the outcome of the check was archived", 0.52)—but a row whose own predicate *binds* a state to that verdict ("the status flag was set according to the check result") carries it at 0.94 ($n=256$ each, McNemar $p=3\times10^{-33}$, identical upstream and question). A served row thus carries the resolved value of the state its own predicate binds—roughly one bit of it—and nothing it merely mentions, neighbors, or follows; mirroring probe (1)'s polarity-yes / exact-token-no split, now with a task payoff, while higher-entropy specifics and derived facts still require their source rows. This is why the deployed harvest ties rather than beats re-encoded text (§5): agent answers are usually exact values and derived facts, precisely the content sparse serving cannot carry without its source. It is also a construction rule for write-time notes: a note memoizes a conclusion only if its own predicate binds it (a paired in-trajectory note reading "determine the threshold verdict now and record it here"—no answer, no operands—lifts post-drop recovery of a *computed* verdict from 0.54 to 0.70, $n=192$, $p=0.005$, while the same note leaves a three-digit value at exactly 0), whereas a note that merely refers to a conclusion stores nothing.

6 Related work

Agent-memory systems. The systems we compare to—AMA-Agent, Mem0 (Chhikara et al., 2025), MemoryBank (Zhong et al., 2024), A-Mem (Xu et al., 2025), MemGPT (Packer et al., 2023), Zep (Rasmussen et al., 2025), MemoRAG (Qian et al., 2024), HippoRAG (Gutiérrez et al., 2024)—all *construct* something at write time with an LLM (facts, notes, summaries, graphs, a compressed

global memory), and most also keep the raw history; they differ in what the reader is served. Mem0 retrieves only its extracted facts; MemoRAG and HippoRAG use their constructed memory or graph to retrieve raw passages; AMA-Agent's released code serves its top-5 raw turns by embedding similarity together with an LLM-written state summary, and falls back to keyword search over the raw trace. AMA-Bench's own needle control prices serving the constructed form: replacing the raw answer-bearing turns with MemoryBank's constructed memory drops accuracy from 0.46 to 0.27 (-41% relative, BabyAI). Two precedents are closest to us. *Generative Agents* (Park et al., 2023) retrieves by recency-importance-relevance over a stream that mixes verbatim observations with summarized reflections, but gives no verbatim-rehydration guarantee. *MemGPT* (Packer et al., 2023) pages verbatim context via LLM tool calls but recursively summarizes overflow rather than keeping it exactly rehydratable. VeraKV differs by constructing no LLM-written memory at all: raw spans are the only evidence payload, routed to by a cheap index, with extractive gists serving only as a one-line index and overview scaffold—never as a substitute for evidence a query needs, which is always rehydrated verbatim. Crucially, all of these are query-aware, so query-awareness is not the novelty, and neither is serving raw spans as such (AMA-Agent does too); the combination of a *verbatim-only payload* + *cheap routing* + *gist-as-index* is.

Raw-history memory: an established design pattern. Serving the raw history from a cheap index is by now a settled pattern, and we claim no priority on it. ReadAgent (Lee et al., 2024) navigates by gists and looks up original passages on demand; Memex (Wang et al., 2026) and HORMA (Hsu et al., 2026) keep full-fidelity traces in databases indexed by summaries (with RL-trained retrieval); EMBER (Li et al., 2026c) retains budgeted verbatim evidence capsules; OCR-Memory (OCR-Memory authors, 2026) predicts segment indexes optically and returns the original text deterministically; EM-LLM (Fountas et al., 2024) segments streams into episodic events at the KV level; filesystem-backed agents search stored transcripts with tools (Letta Team, 2025; Pam Research, 2025), as does AMA-Agent's tool loop on agent traces. Closest of all, and concurrent, An (2026) run the controlled comparison directly—fixed pipeline, verbatim chunks vs. LLM-extracted typed artifacts—and report $15.9/22.0\,\mathrm{pp}$ gaps on LOCOMO/LongMemEval-S. Our Part I is therefore an *extension and a decomposition*, not a discovery claim: we bring the controlled comparison to dense *agent trajectories*, add the original-line control that separates *generative paraphrase* from *extractive shortening* (the artifact-vs-chunk contrast conflates them), add an oracle-evidence variant and a different-family judge control, and pair it with the memory–reader decoupling. Concurrently, MemDelta (MemDelta authors, 2026) makes the confound-control case for agent-memory *evaluation*—varying one pipeline component at a time on LongMemEval and showing rankings reverse; our matched-evidence protocol is the payload-form instance of the same discipline, and we adopt their recommendation to report memory-only and end-to-end numbers separately (§4.1).

Retrieval-augmented generation and agent frameworks. Classic RAG retrieves external passages and conditions the reader on them—RAG (Lewis et al., 2020), REALM (Guu et al., 2020), RETRO (Borgeaud et al., 2022), Atlas (Izacard et al., 2023). VeraKV is RAG turned *inward*: the corpus is the agent's own trajectory and the retrieved units are re-injected as *verbatim KV* rather than re-encoded text, so retrieval and reuse share one representation (the "standard vector-RAG" point of this design is our EmbeddingRouter, §4.4). The memory modules practitioners actually deploy—the buffer/summary/vector stores in LangChain (Chase, 2022) and AutoGen (Wu et al., 2023)—occupy exactly the slot VeraKV targets; it is a drop-in, verbatim-first alternative to those summary/vector defaults.

KV reuse and compression. A systems line reuses KV caches for latency—prefix caching in vLLM (Kwon et al., 2023) and SGLang (Zheng et al., 2024)—which VeraKV sits on top of. A compression line removes tokens via eviction (e.g. H2O (Zhang et al., 2023)) or learned summarization (RECOMP (Xu et al., 2024)); IntentKV (Li et al., 2026a) prunes agent KV across turns with a learned intent signal; these optimize the cache or the prompt, not the *selection-quality* memory question that determines whether a multi-hop answer survives. Reuse of rollout KV also appears beyond language agents—WorldKV (Yi et al., 2026) re-inserts evicted cache chunks into the native attention window of a video world model without re-encoding—and, concurrently with this work, Li (2026) show precompiled *skills* are position-portable: RoPE-repositioned KV splices are near-indistinguishable from full recompute across twelve models. Our skill-block result (§4.6) is an application of that transplantation idea to the agent-memory answering protocol, validated at

answer-accuracy parity on official-scale memory QA; the mechanism is theirs, the memory-side characterization ours. A parallel architecture line compresses or retrieves *past activations* inside the model—Compressive Transformer (Rae et al., 2020), Memorizing Transformer (Wu et al., 2022)—whereas VeraKV sits one level up, as a training-free memory manager over the raw KV of a frozen model. Our point is orthogonal and, where they overlap (at a matched training-free, query-free budget), extractive retention is already at least as good as generative summary—so the lever is selection, not harder compression.

Position-independent KV caching. A recent serving line moves beyond prefix caching toward reusing recurring *spans*—documents, code files, retrieved chunks—even when they appear at different prompt positions, which standard prefix caching cannot do (it reuses KV only across an identical shared prefix, and only in the prefill). EPIC (Hu et al., 2024) formalizes position-independent caching and recovers accuracy near segment boundaries; MiniPIC (Ordonez & Parnell, 2026) supports it in a minimal vLLM substrate by storing *position-free* keys and applying RoPE at attention time under per-request logical positions; MEPIC (Wang et al., 2025) makes reused chunk blocks shareable across positions and requests to cut HBM; and HYPIC (Liu et al., 2026) extends the idea to hybrid-attention models. Earlier, Prompt Cache (Gim et al., 2023) reused modular attention states and CacheBlend (Yao et al., 2024) showed that non-prefix chunk KV must be partially recomputed to stay faithful. VeraKV is complementary and *upstream*: these systems assume the reusable spans are given and optimize their serving, whereas VeraKV decides *which* spans of a long agent trajectory to select and shows a verbatim payload beats a lossily constructed one. Our KV serving is also deliberately restricted—it is *position-preserving* (spans stay at their original trajectory positions, exact for RoPE) rather than position-independent, so it does not compact the resident KV; a memory-compact version that relocates spans would need exactly the PIC mechanisms above (position-free keys, logical positions, block/page sharing), which we do not claim to contribute.

Repairing spliced caches. When chunks are encoded independently and later spliced, a repair family restores quality: anchor-block conditioning (Star Attention (Acharya et al., 2024)), partial recomputation of high-deviation or boundary tokens (CacheBlend (Yao et al., 2024), EPIC (Hu et al., 2024)), and sparse-query selection with recency fallbacks over segment-level reuse (SparseX (MemTensor, 2026), the nearest neighbor to §5; it also reports the naive-reuse failure and a repair on memory benchmarks); KEEP (Yang et al., 2026) moves group-level recompute selection by query-conditioned attention propagation into an embodied-planning memory system. In RAG-style regimes— self-contained chunks, all chunks served—this family reports repair $\approx$ full re-prefill. §5 instantiates the anchor and recompute families as agent-memory operations under routed serving and finds the parity claims do not transport: the boundary-recompute budget that suffices for RAG chunks is a null on agent events, whose isolation damage is distributed across the whole event (§5, Regularity II), and KEEP's propagation selector, run same-harness at matched budget, lands on the random floor below recency. Our contribution relative to this family is the paired official-benchmark characterization and the two regularities, not the primitives or their combination.

7 LIMITATIONS: WHEN DOES VERBATIM SELECTION WIN?

VeraKV defers answer-extraction to the reader at inference time, which draws a clear regime boundary we chart rather than hide. Verbatim selection wins when the answer is an *exact, objective span buried in dense history* and the history either overflows the window or drowns the reader in distractors—the setting of AMA-Bench and, on dialogue, LOCOMO. It does not dominate everywhere: when the whole history *fits* the window and the reader can exploit it, full context can win (LongMemEval-S at 128k, $+6.8\,\mathrm{pp}$ at $4.5\times$ the budget); when the reader is *weak* and the answers are tiny isolated facts, systems that extract atomic facts at construction time can present an easier context. Both crossovers are in our results (§4.3), and both are about reader capacity and history length, not retrieval—on LongMemEval-S only 9% of the routed arm's answerable misses are selection failures (§4.3).

The two pillars have distinct boundaries. The *KV* pillar's strongest form—position-preserving subselection of arbitrary spans—needs a self-hosted reader; hosted APIs expose only contiguous-prefix prompt-caching, which our prefix-friendly layout still benefits from but which cannot gather arbitrary spans. We therefore report the KV serving as a scoped prototype (§4.7; its serving regimes and

their trajectory-length limits are in Table 1), and report LongMemEval-S only in the strong-reader (Qwen3-32B) regime—a scoping choice, testing the settings the claim is about.

The *select* pillar's one visible weak spot is AMA-Bench SOFTWARE. There selection achieves 100% recall of the *named* steps yet AMA-Agent's published score leads by $+17\,\mathrm{pp}$; the dissection in Appendix E eliminates retrieval, tool access, output format, and prompted iteration, and localizes the residual to *temporal-scan* questions whose answer step is unnamed, where even the gold step supplied verbatim leaves a reader at ~ 0.12. We first read this as a coverage ceiling, but an audit of the labels does not support that reading alone: for most questions asking when something was first or last modified, the gold step is not an edit (Appendix E). We make no claim that more coverage, or a construction-time state summary, would close this gap.

Finally, verbatim storage carries governance costs, as does any system that also keeps the raw history—which many constructed memories do. The store retains raw history: PII persists verbatim and must be handled by policy outside the memory (encryption at rest, retention windows, write-time scanning). Deletion semantics differ by layer. In the *text* substrate, dropping a turn is exact—the next assembly simply omits it. In the *KV* layer it is not: cached keys/values are contextualized, so the KV of tokens *after* an erased turn was computed while attending to it, and exact erasure requires invalidating or recomputing the downstream blocks (bounded, in our per-episode caches, by the episode length), or learned steering states that localize the edit without a full recompute (Li et al., 2026b). We measured whether that residue is *extractable*: planting a secret at one step, erasing it, and serving only later steps, the gathered-KV arm emitted the secret in $0/120$ trials and its first-token log-probability sat at the noise floor of an arm that never saw it (≈ -31 nats), while a positive control leaks $(120/120)$—so the carryover is measurable at the logit level but did not surface erased content in generation; a functional-erasure probe (§5) shows the same for state, not just secrecy—with a deleted value's rows removed, generation matches an oracle that never saw it $(0/96)$. We still treat exact KV-layer erasure as requiring downstream invalidation, make no claim under adversarial extraction, and have not evaluated redaction pipelines.

The out-of-window study (§5) carries its own scope. Its conditioner and geometry dials and fresh-selector comparisons are 8B-only (n=824) and presumed, not shown, to transfer to 32B (the evidence-matching audit, the harvest cell, and the skill block do replicate at 32B and on LOCOMO, and the harvest cell also on a Llama backbone); the non-inferiority margin was not pre-registered; the within-window 32B edge has a clustered CI touching zero; results beyond those replications cover one model family (Qwen3) and uncompacted position-preserving stores whose per-token storage economics we do not measure; and end-to-end latency inverts for long answers, so the cache path claims reduced TTFT only.

8 CONCLUSION

For a long-horizon agent whose answers are exact, objective, and buried in a dense trajectory—and whose history overflows the window or drowns the reader in distractors—the right memory operation is to *deliver evidence faithfully*: construct indices, not answer payloads. VeraKV keeps the raw history as reusable KV, indexes and scaffolds it cheaply, selects the spans a query needs, and serves them verbatim—within the window, from their cached KV, a carrier computationally faithful to the model's own full-prefill reading and accuracy-equivalent to re-encoded text at lower first-token cost. When the history fits and the reader can exploit it, full context can win; when the reader is weak and the facts are tiny, construction can win; we state both boundaries. Within its regime, a small framework built on this principle leads our runs of AMA-Bench (submitted and leaderboard-verified as the top entry, #1 as of 2026-07-15), is competitive with dedicated dialogue-memory systems on LOCOMO, and traces the selection-vs-full-context crossover on LongMemEval-S—with the win driven by payload fidelity rather than router sophistication, the reader treated as a separable axis whose answering protocol the memory can itself serve, and the cost amortized by KV reuse. Beyond the window the same principle survives in a weakened, priced form: stores must be written with a conditioning that matches how they will be served—the true opening, at its true position—and served lean with a small fresh tail; what cannot be bought back at any serving budget is the joint re-encoding a strong reader performs when it re-reads. And where the agent's own rollout cache survives, even the write is unnecessary: the memory harvested from that cache, with the answering protocol riding along as a rotated skill block, ties the deployed text pipeline on the full official

harness. The recipe is simple enough to fork, and we release it, with configurations and predictions included, as a reproducible baseline that centers payload fidelity over memory construction.

REPRODUCIBILITY STATEMENT

All code, configurations, the exact answer instructions (Appendix B), raw per-question predictions with judge scores for every AMA-Bench run reported here (26 tagged runs, indexed by a results manifest), the KV-equivalence and planted-secret probes with their outputs, the analysis scripts that reproduce the bootstrap/cluster CIs and the step-pin splits, and our leaderboard submission file are released at `https://github.com/oklen/VeraKV`. AMA-Bench numbers come from the official `run.py` harness with the memory plugged in through its method interface; the structured answer instruction is a config-gated one-line swap, disclosed and ablated in §4.4.

REFERENCES

Shantanu Acharya, Fei Jia, and Boris Ginsburg. Star attention: Efficient LLM inference over long sequences. *arXiv preprint arXiv:2411.17116*, 2024.

Tao An. Verbatim chunks beat extracted artifacts: A controlled ablation of memory representations for long LLM conversations. *arXiv preprint arXiv:2601.00821*, 2026.

Sebastian Borgeaud et al. Improving language models by retrieving from trillions of tokens. In *International Conference on Machine Learning (ICML)*, 2022. arXiv:2112.04426.

Harrison Chase. LangChain: Building applications with LLMs through composability, 2022. Software, github.com/langchain-ai/langchain.

Prateek Chhikara et al. Mem0: Building production-ready ai agents with scalable long-term memory. *European Conference on Artificial Intelligence (ECAI)*, 2025. arXiv:2504.19413.

Zafeirios Fountas, Martin A Benfeghoul, Adnan Oomerjee, Fenia Christopoulou, Gerasimos Lampouras, Haitham Bou-Ammar, and Jun Wang. Human-like episodic memory for infinite context LLMs. *arXiv preprint arXiv:2407.09450*, 2024.

In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. *arXiv preprint arXiv:2311.04934*, 2023.

Bernal Jiménez Gutiérrez et al. HippoRAG: Neurobiologically inspired long-term memory for large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2405.14831.

Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. REALM: Retrieval-augmented language model pre-training. In *International Conference on Machine Learning (ICML)*, 2020. arXiv:2002.08909.

Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. RULER: What's the real context size of your long-context language models? *arXiv preprint arXiv:2404.06654*, 2024.

Hao-Lun Hsu, Nikki Lijing Kuang, Boyi Liu, Zhewei Yao, and Yuxiong He. Organize then retrieve: Hierarchical memory navigation for efficient agents. *arXiv preprint arXiv:2606.11680*, 2026.

Junhao Hu, Wenrui Huang, Weidong Wang, Haoyi Wang, Tiancheng Hu, Qin Zhang, Hao Feng, Xusheng Chen, Yizhou Shan, and Tao Xie. EPIC: Efficient position-independent caching for serving large language models. *arXiv preprint arXiv:2410.15332*, 2024.

Gautier Izacard et al. Atlas: Few-shot learning with retrieval augmented language models. *Journal of Machine Learning Research (JMLR)*, 2023. arXiv:2208.03299.

Woosuk Kwon et al. Efficient memory management for large language model serving with PagedAttention. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2023.

Kuang-Huei Lee, Xinyun Chen, Hiroki Furuta, John Canny, and Ian Fischer. A human-inspired reading agent with gist memory of very long contexts. *International Conference on Machine Learning (ICML)*, 2024.

Letta Team. Benchmarking AI agent memory: Is a filesystem all you need? `https://www.letta.com/blog/benchmarking-ai-agent-memory/`, 2025. Filesystem-backed agent memory over raw transcripts; reports LOCOMO 0.740 with gpt-4o-mini.

Patrick Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. arXiv:2005.11401.

Bojie Li. Models take notes at prefill: KV cache can be editable and composable. *arXiv preprint arXiv:2606.17107*, 2026.

Junjie Li, Jiong Lou, and Jie Li. IntentKV: Cross-turn intent-aware KV cache pruning for agent inference. *arXiv preprint arXiv:2606.09916*, 2026a.

Mufei Li, Shikun Liu, Dongqi Fu, Haoyu Wang, Yinglong Xia, Hong Li, Hong Yan, and Pan Li. KVEraser: Learning to steer KV cache for efficient localized context erasing. *arXiv preprint arXiv:2606.17034*, 2026b.

Yilong Li, Suman Banerjee, and Tong Che. EMBER: Efficient memory via budgeted evidence retention for long-horizon agents. *arXiv preprint arXiv:2606.05894*, 2026c.

Yifei Liu, Juntong Wu, Yang Liu, Junhao Hu, Minghao Li, Xiaoxu Chen, and Weihang Chen. HYPIC: Accelerating hybrid-attention LLM serving with position-independent caching. *arXiv preprint arXiv:2607.01299*, 2026.

Adyasha Maharana et al. Evaluating very long-term conversational memory of LLM agents. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. arXiv:2402.17753.

MemDelta authors. MemDelta: Controlled baselines and hidden confounds in agent memory evaluation. *arXiv preprint arXiv:2606.29914*, 2026.

MemTensor. SparseX: Efficient KV-cache reuse across multi-turn, RAG, and multi-agent workloads. *arXiv preprint arXiv:2606.01751*, 2026.

OCR-Memory authors. OCR-Memory: Optical context retrieval for long-horizon agent memory. In *Proceedings of ACL*, 2026.

Nathan Ordonez and Thomas Parnell. MiniPIC: Flexible position-independent caching in $<$100LOC. *arXiv preprint arXiv:2606.13126*, 2026.

Charles Packer et al. MemGPT: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

Pam Research. Pam outperforms ChatGPT by 40% on LoCoMo benchmark. `https://manager.harmix.ai/blog/pam-locomo-benchmark-results`, 2025. File-first memory; reports LOCOMO 0.744.

Joon Sung Park et al. Generative agents: Interactive simulacra of human behavior. In *ACM Symposium on User Interface Software and Technology (UIST)*, 2023. arXiv:2304.03442.

Hongjin Qian et al. MemoRAG: Moving towards next-gen rag via memory-inspired knowledge discovery. *arXiv preprint arXiv:2409.05591*, 2024.

Qwen Team. Qwen3 technical report. In *arXiv preprint*, 2025. arXiv:2505.09388.

Jack W. Rae et al. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations (ICLR)*, 2020. arXiv:1911.05507.

Preston Rasmussen et al. Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2025.

Qian Wang, Zahra Yousefijamarani, Morgan Lindsay Heisler, Rongzhi Gu, Bai Xiaolong, Shan Yizhou, Wei Zhang, Wang Lan, Ying Xiong, Yong Zhang, and Zhenan Fan. MEPIC: Memory efficient position independent caching for LLM serving. *arXiv preprint arXiv:2512.16822*, 2025.

Zhenting Wang, Huancheng Chen, Jiayun Wang, and Wei Wei. Memex(RL): Scaling long-horizon LLM agents via indexed experience memory. *arXiv preprint arXiv:2603.04257*, 2026.

Di Wu et al. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.10813.

Qingyun Wu et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

Yuhuai Wu, Markus N. Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *International Conference on Learning Representations (ICLR)*, 2022. arXiv:2203.08913.

Fangyuan Xu, Weijia Shi, and Eunsol Choi. RECOMP: Improving retrieval-augmented lms with compression and selective augmentation. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.04408.

Wujiang Xu et al. A-Mem: Agentic memory for llm agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2502.12110.

Zebin Yang, Tong Xie, Baotong Lu, Shaoshan Liu, Bo Yu, and Meng Li. KEEP: A KV-cache-centric memory management system for efficient embodied planning. *arXiv preprint arXiv:2602.23592*, 2026.

Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. *arXiv preprint arXiv:2405.16444*, 2024.

Jung Yi, Minjae Kim, Paul Hyunbin Cho, Wooseok Jang, Sangdoo Yun, and Seungryong Kim. WorldKV: Efficient world memory with world retrieval and compression. *arXiv preprint arXiv:2605.22718*, 2026.

Zhenyu Zhang et al. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2306.14048.

Yujie Zhao, Boqin Yuan, Junbo Huang, Haocheng Yuan, Zhongming Yu, Haozhou Xu, Lanxiang Hu, Abhilash Shankarampeta, Zimeng Huang, Wentao Ni, Yuandong Tian, and Jishen Zhao. AMA-Bench: Evaluating long-horizon memory for agentic applications. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2602.22769.

Lianmin Zheng et al. SGLang: Efficient execution of structured language model programs. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2312.07104.

Wanjun Zhong et al. MemoryBank: Enhancing large language models with long-term memory. In *AAAI Conference on Artificial Intelligence*, 2024. arXiv:2305.10250.

A PROTOCOL LEDGER: EVERY HEADLINE NUMBER, ITS PIPELINE, AND ITS BATCH

The paper reports numbers from *four* distinct pipelines (deployed text, within-window KV gather, out-of-window stores, mixed deployment) and several same-batch subsets. Table 10 maps each headline number to its exact protocol so that no cross-pipeline comparison can be made by accident; the notes below resolve the specific pairings a careful reader will ask about.

Table 10: Ledger of headline numbers. "Text" = deployed text pipeline (§2); "cell" = the out-of-window serving-cell protocol (fixed-K retrieval, no overview, official default answer prompt, official judge); reader is Qwen3-32B unless stated. CIs where shown are episode-clustered.

Number	What	Pipeline	Answer prompt	n (clusters)	Run tag / locus
0.6478	submitted entry, leaderboard #1 (as of 07-15)	text	structured	2496 (208)	FLSA+B; Tab. 3
0.6466	deployed router+pin, local run; lexical+pin, run separately, scores the same	text	structured	2496 (208)	RESTRF3, Tab. 6; RESTRLEXF, App. F
0.6410	model-pick hybrid base + pin (audit)	text	structured	2496 (208)	App. F
0.5954	memory-only headline (CI [.570, .620])	text	default	2496 (208)	FPA+B
0.5601/0.6154	factorial lexical cells	text	default/struct.	2496 (208)	R0/R1
0.6290/0.6418	tab:7 same-day anchors	text	structured	1248 (104)	two half-set batches
0.5577	no-memory recency-truncated arm	text	structured	1248 (104)	Tab. 7
.397/.265/.360	serving cell, K=12	cell	official default	2431 (208)	Tab. 9
.368/.360	serving cell, K=5	cell	official default	2403 (208)	Tab. 9
.3965	deployment map (gsk)	mixed	protocol-as-skill	2431 paired	§5
.2209 etc.	mechanism subset dials	cell (8B)	stripped, 64 tok	824 (103)	§5
17.3/17.4%	KV-vs-text end-to-end parity	gather (8B)	stripped	896 (112)	§4.5
LOCOMO .47–.64	KV cell, sessions as events	cell (8B/32B)	short-answer	1150 (10)	§5

Notes. (a) *The champion family.* 0.6478 is the submitted entry as self-scored and subsequently leaderboard-verified. Re-runs of the submitted configuration (deployed router + step-pin) span 0.6454–0.6538, consistent with the stated ± 0.5 pp same-day replicate noise plus cross-day drift, and 0.6410 after a serving-stack upgrade; one re-run scores 0.6466. The released lexical+pin default, run separately, also scores 0.6466, the number we quote for it. The submitted entry's base router fused a model-pick signal that the audit (App. F) finds score-neutral; the $+1.4$ pp attributed to it in early development was measured before that audit and is superseded by it. (b) *Serving-cell denominators.* n=2431 at K=12 (and 2403 at K=5) is the three-arm *judged intersection*: the remaining questions of the 2,496 belong to episodes lost to out-of-memory failures in at least one arm's serving run and are excluded from *all* arms symmetrically; the deployment-map evaluation pairs against exactly the same 2431-question set. (c) *The serving cell is not the deployed pipeline.* It uses fixed-K retrieval, no overview, and the official default answer prompt, so its text arm (.397) is a routed-text *carrier* baseline, some 25 pp below the deployed pipeline of Table 3—the two must never be compared across protocols. (d) *Half-set anchors.* Table 7 reports two same-day anchor batches (0.6290/0.6418); every Δ in that table is computed within its own batch, never across. (e) *Mechanism subset.* All §5 dial experiments run on a fixed 103-episode subset (n=824) with a stripped 8B harness; these episodes are contained in the full official set, so full-set evaluations that follow subset-driven development are labeled accordingly (§3).

B THE THREE ANSWER INSTRUCTIONS, VERBATIM

The reader-side variable of §4.4 is a single swapped sentence. For auditability we reproduce all three instructions exactly as run.

Default (harness).

> Provide a direct and concise answer.

Structured (ours; $+5.1$–5.5 pp at either router).

> Answer using ONLY the context above. Break the request into sub-questions if needed, answer each citing exact evidence (values, step indices, outcomes) VERBATIM, then give the final answer.

Quote-first (probe; -10.0 pp on SOFTWARE, same batch).

> Answer using ONLY the context above. Locate the exact evidence first, then answer by COPYING the exact strings from the context VERBATIM—commands, file paths, code, values, error messages, step indices. Do not paraphrase or re-describe them. The final answer should be the exact string(s) plus the minimum wording needed to address the question.

The comparison is itself a control: the structured and quote-first instructions share the grounding preamble ("using ONLY the context above") and the same emphasis on verbatim evidence, and differ in what they ask the reader's *reasoning* to do—work through sub-questions versus copy strings. The former gains $+5$–6 pp overall, the latter loses 10 pp on its probe domain: the structured instruction's

value is the elicited sub-question reasoning in the answering pass, not the grounding phrase or the verbatim emphasis it shares with a losing variant.

C THE READER-SIDE FALSIFICATION MAP

This appendix carries the mechanism-level evidence behind the reader-saturation finding of §4.4; Table 11 tabulates all fourteen mechanisms. On the flagship memory we tested: letting the model compress the selected evidence before answering; a sufficiency gate fused into the answer prompt that can trigger re-retrieval; the same gate as a separate call; a query-conditioned entity-state timeline prepended as a scaffold; and deterministic lookup tools (exact get-by-index, exhaustive search) over the lossless store. None of these helps; several hurt, and the attribution is exact: the fused gate's -1.6 pp splits into -2.6 pp from the escape clause perturbing the answer prompt and $+0.4$ pp from the re-retrieval action itself.

Four further probes sharpen this picture. *First, the instruction's content cannot be pre-computed.* If a one-sentence instruction that elicits sub-question reasoning is worth $+5$ pp, can the memory service cache that reasoning—produce an evidence plan (sub-questions, cited values, local conclusions) and hand it to a plain reader? No: prepending such a plan costs -5.6 pp under the default reader and -2.7 pp under the structured one (same-batch)—*worse than no scaffold at all*—and the more reasoning a scaffold carries, the more it hurts (plan -5.6 vs state timeline -1.6; compression's -3.0 removes context outright, a different failure). The reader inherits the plan's errors and skips its own derivation: the instruction's value is eliciting the reader's *own* reasoning in the answering pass, not the reasoning content. *Second, the reasoning's presentation cannot be compiled.* We compiled the same selected verbatim evidence into a reasoning-affordant view—need-grouped quotes, occurrence lists, timelines, conflict and missing-evidence panels, with conclusions and instructions expressly forbidden—and handed it to the *default* reader, targeting exactly the questions the structured instruction fixes (default-wrong, structured-right; n=316), with a same-batch raw-context control and a 299-question both-right stability sample. The view recovers nothing beyond the control's spontaneous-recovery rate (32.6% vs 32.3%; a same-config re-run alone re-fixes a third of the "fixed" set, so per-question flips are dominated by borderline items) and *breaks* 13.4 pp of previously-correct questions, through mechanisms we verified against raw trajectories: appendix-scoped absences asserted as data-wide facts, and fabricated citations—one audited compilation promoted an environment's *available-actions menu* to executed events, with step numbers and quote marks—which the reader trusts over the overview it still has. A hardened second iteration (subset-scoped wording, computed deltas and absence claims forbidden, an explicit option-vs-event rule, a deterministic verbatim-containment guard) still saw the guard reject 69% of compilations—instructions alone do not enforce fidelity—and, deployed on the full 1,248-question half, is exactly null against its same-batch control (0.5617 vs 0.5641 at a 31% serve rate): harmless, and worthless. *Fidelity can be guarded; usefulness cannot be compiled.* In hindsight the boundary could not have held: grouping asserts relevance, a timeline asserts change, a missingness note asserts nonexistence—every panel of a compiled view is an implicit conclusion, so a view compiler is a query-time lossy *constructor*, and the paper's core claim applies to it recursively (the released paired case study shows wins and losses flowing through the same mechanisms).

Third, the instruction's parts cannot be specialized by question type. Deterministic routing to purpose-built fragments of it (enumerate-then-count, ledger-only state reconstruction, verbatim-copy, protective direct answering—rules derived from an annotated 26-case study of what the instruction fixes and breaks) scores -3.0 pp overall: the routed classes lose -7 to -23 pp while the untouched-default remainder replicates its anchor (-0.3 pp). The routing *direction* was right—on the questions the instruction breaks, routed fragments gain $+10.7$ pp—but the exchange rate is ruinous: within the same classes, the questions the instruction *fixes* lose -39 pp, about ten collateral losses per targeted fix. Every fragment ablates the full derive→cite→compose loop, which is exactly how the instruction was quietly winning the body of each class; no rule precision fixes a treatment whose side effect is removing the cure. (We did not test additive per-class hints appended to the intact instruction.) *Fourth, a repair mechanism needs something specific to fetch.* Re-retrieval cannot fix *breadth* starvation: at a 6k budget (a known -9.5 pp gap), a separate sufficiency gate fires on 19% of questions yet recovers nothing (-0.4 pp)—the budget cut removes the overview, and fetching more individual steps does not restore breadth.

The trust relay: repair versus override. The one composed mechanism that meets the single structured pass reuses the memory service's own structured analysis: the reader is told to verify that analysis and answer *in full*, reaching 0.663 on the full $2{,}496$-QA set ($+1.7$ $[+0.1, +3.3]$ over its same-batch structured anchor). It retains the structured pass's wins and *repairs* part of what that pass breaks; the residual losses are independent re-adjudications, not compression (78–95% are overrides despite the trust instruction), and the same re-judgment buys the repairs—a full-set audit prices the exchange at 6.7% of questions repaired against 4.5% destroyed. Requiring the reader to *name the contradicting step* before overriding raises override precision to 0.672 (vs 0.645; 0.675 on the half, $+4.0$ $[+1.5, +6.5]$)—the strongest configuration we measure, indifferent to the router base. It sits above the submitted single-pass system but is an internal exploration, not part of the leaderboard entry.

Table 11: Reader-side mechanisms over a *fixed* flagship memory (AMA-Bench, same-batch Δ vs the plain single-pass reader; the plan row reports both readers: default / structured). Same-day replicate noise is ± 0.5 pp; the tools and loop rows are probed on the SOFTWARE domain, where the mechanisms were hypothesized to matter and whose batch noise is ± 4 pp—so $+0.7$ reads as ≈ 0; both loop rows are same-batch against their own control. The protocol rows are measured against same-batch *default* anchors on two independent full-set runs (§the serveable-lever paragraph). Only re-retrieval under a weak router helps—the one case where the mechanism supplies evidence the selected context is missing; the served protocol is the constructive exception this pattern predicts.

Reader-side mechanism (memory fixed)	Δ accuracy
Model-selected evidence compression	-3.0 pp
Entity-state timeline scaffold	-1.6 pp
Sufficiency gate fused into the answer prompt	-1.6 pp
(escape clause alone -2.6 pp; re-retrieval action alone $+0.4$ pp)	
Sufficiency gate as a separate call	± 0.0 pp
Deterministic lookup tools over the lossless store	$+0.7$ pp
Iterative lookup loop *replacing* the answering pass	-9.0 pp
Iterative lookup loop as evidence-gatherer (answer pass kept)	± 0.0 pp
Type-routed specialized instructions (rules from a 26-case study)	-3.0 pp
Pre-computed reasoning plan (memory-side artifact)	-5.6 / -2.7 pp
Re-retrieval under budget starvation (6k cap, gate fires 19%)	-0.4 pp
Requirements checklist appended to the *context* (instruction untouched)	-1.8 pp
Verbatim-evidence worksheet appended (containment guard rejects 74%)	-0.7 pp
Answering protocol served in the retrieved context (directives only)	$+2.3$ / $+3.8$ pp
+ two schematic worked examples (question substitution)	-9.6 pp
+ hardened emission wording (precedence, length anchor)	-1.7 pp
Re-retrieval under a *weak* (lexical) router	$+1.3$ pp

D MEMORY ASSEMBLY: PER-DOMAIN SELECTION-VS-OVERVIEW SPLIT

Table 12: Memory assembly is second-order at fixed budget. Crossing routed verbatim *selection* $\times$ a one-line *gist overview* of the non-selected turns, same batch (AMA-Bench `openend`, n=2496, Qwen3-32B, structured reader, lexical router, 22k budget). All four cells lie within 0.9 pp and are pairwise indistinguishable (paired McNemar $p > 0.28$). The overview is a navigation scaffold: adding it to recency moves accuracy by -0.4 pp (n.s.), with or without selection. The routed appendix adds little *at this weak router*; its value grows with selection quality (-4.3 pp to remove at the deployed router+pin, Table 7).

Routed verbatim selection	Gist overview	Overall	SOFTWARE
–	–	0.610	0.454
–	✓	0.604	0.438
✓	–	0.611	0.482
✓	✓	0.613	0.479

The 2×2 of Table 12 is near-neutral *on average*, but the two non-trivial arms trade domains rather than tie everywhere. Table 13 breaks the selection-only (retrieval) minus overview-only (gist) contrast by domain (both at the lexical router, structured reader, same batch, n=2496). Routed verbatim

selection helps the retrieval-heavy domains—SOFTWARE ($+4.4$ pp), Web ($+3.2$ pp), TEXT2SQL ($+2.0$ pp)—where the answer lives in a few locatable steps and a one-line gist of them loses the needed detail. It *loses* on the holistic domains, Game (-5.3 pp) and OpenWorld (-2.5 pp), where the question ranges over the whole trajectory and a complete (if lossy) overview beats a handful of verbatim steps. This is the same regime axis as the router and Limitations analyses: selection wins when the evidence is addressable, a broad overview wins when it is diffuse. The deployed system keeps both precisely because the win is domain-dependent.

Table 13: Selection-only minus overview-only (retrieval $-$ gist) by domain (lexical router, structured reader, same batch, n=2496). Positive = routed verbatim selection beats a full gist overview.

Domain	Overview-only	Selection-only	Δ
SOFTWARE	0.438	0.482	$+4.4$ pp
Web	0.661	0.694	$+3.2$ pp
TEXT2SQL	0.652	0.672	$+2.0$ pp
Embodied AI	0.626	0.629	$+0.3$ pp
OpenWorld QA	0.658	0.633	-2.5 pp
Game	0.589	0.536	-5.3 pp

E SOFTWARE TEMPORAL-SCAN: UNIFORMLY LOW, NOISILY LABELLED

SOFTWARE is the one domain VeraKV trails (Table 3), and its failures concentrate in *temporal-scan* questions—"which step *first* / *last* did X"—which are 19% of the domain and where *every* memory configuration scores ~ 0.15 (routed selection, plain recency, and a full gist overview alike; the overview is in fact worst, 0.06, because one-line gists lose the detail needed to pin an occurrence). This is not a failure on *named* evidence: when a question names its step, selection recalls it (100% of asked steps) and adding deterministic lookup tools over the lossless store leaves accuracy unchanged ($+0.7$ pp)—the residual is a different kind of question. To separate retrieval from reasoning we hold the reader fixed and vary only the evidence, on the 74 temporal-scan questions whose gold answer names a clean step (Table 14): the deployed *selection*; selection *plus the gold step* verbatim; and a complete one-line *index of every step* plus the gold step verbatim. Two facts stand out. First, the answer-bearing step reaches the reader only 5.4% of the time under budget-limited selection—identical at both reader sizes, because it is a property of the router, not the reader. Second, *supplying* that step verbatim lifts accuracy only to ~ 0.12 on *both* an 8B and a 32B reader, and a complete (lossy) index does not help further. We first read this as a coverage ceiling: answering "which step first" requires confirming that *no other* step qualifies—the entire 60–100k-token trajectory (median 94 steps) held verbatim, which exceeds any fixed budget. An audit of the labels does not support that reading alone. Of the 34 first/last questions that ask when something was modified, changed, or fixed, the gold step is an edit in only 10 (19 are command runs, 5 file views), and none of four gold steps checked by hand modifies the code the question names. So these questions are low for every configuration and noisily labelled; we do not call the residual a coverage ceiling. (Levels sit below Table 3: a clean-gold subset, plain answer prompt.)

Table 14: Software temporal-scan: reader fixed, evidence varied, 74 first/last questions with a clean gold step. Supplying the gold step verbatim reaches only ~ 0.12 on both readers; many gold labels do not match the trajectory (see text), so we do not read this as a coverage ceiling.

Evidence given to the (fixed) reader	Qwen3-8B	Qwen3-32B
Deployed selection (recent + routed)	0.014	0.081
+ gold step verbatim	0.122	0.122
+ complete 1-line index + gold verbatim	0.162	0.135
gold step already present in the selection	5.4%	

F THE DEPLOYED ROUTER: A POST-HOC AUDIT

Preparing the release we found that the "hybrid" router in the deployed configuration fuses lexical overlap with a second *model-pick* signal (a short LLM call that names likely-relevant turns)—*not* the dense-embedding router used on LOCOMO and LongMemEval-S. The harness leaves the model's thinking mode on (it never passes its `enable_thinking` setting), so each 32-token pick call returns only the opening of an unfinished reasoning block (all 7,128 such calls logged in two later re-runs), and the parser reads whatever numbers appear there as turns. We audited whether the signal contributes at all.

It does not. A same-day paired audit on the half set put a plain lexical base *ahead* ($+2.7$ pp $[+0.5, +5.0]$); a pre-registered full-set replication is score-neutral (-0.8 pp $[-2.3, +0.8]$, the two bases trading domains; the lexical run scores 0.6466). A further full-set run of the model-pick hybrid base + step-pin, made a day later on an upgraded serving stack and therefore not paired with these, scores 0.6410, 0.56 pp below the lexical run and within the run-to-run spread of about 1 pp. A pre-registered same-batch re-run on all $2{,}496$ QA then fixed the pick call (thinking off; prompt and 32-token budget unchanged). The deployed configuration as published scores 0.5992 there with the default answer prompt and 0.6486 with the structured one; the fixed signal changes these by $+0.2$ pp $[-1.5, +2.1]$ and $+1.4$ pp $[-0.2, +3.2]$ (neither significant), and against lexical+pin it is $+1.8$ pp $[+0.0, +3.6]$ and $+0.8$ pp $[-0.8, +2.4]$, the two answer prompts disagreeing. No reported *difference* depends on the signal (memory configuration is held constant within each same-batch table). We therefore name, recommend, and release the reproducible **lexical + step-pin** default: identical accuracy, one fewer LLM call per retrieval, no malformed pick calls. This sharpens the conclusion of §4.4—on AMA-Bench the substrate and the structural pin carry the routing, not a semantic scorer—and it is why we release "lexical+pin" as the default, while the tables report the deployed router the runs used.

G ADDITIONAL AMA-BENCH ROBUSTNESS AND ABLATIONS

Table 15 collects the three memory-side deltas of §4.4 side by side, with the reader-side answer instruction listed for scale (and reported separately because it is *not* a memory effect).

Table 15: AMA-Bench ablation deltas (full 2,496 QA, same-batch). The verbatim substrate carries the result; semantic and structural routing refinements are second-order; the reader-side answer instruction—listed for scale, and reported separately because it is *not* a memory effect—is larger than both combined.

Component	Δ Avg. accuracy
Substrate: raw spans vs. constructed memory (cross-system, confounded)	$+10$ to $+35$ pp
controlled (same oracle evidence; Fig. 3)	$+8$ to $+10$ pp
Fused second router signal (superseded by the post-hoc audit)	$+1.4$ pp
Structural step-pin ($+2.5$ pp on the 73% cited-step questions, $+0.0$ else)	$+1.7$ pp
Reader-side, for scale: structured answer instruction	$+5.1$–5.5 pp

Is the step-pin result a step-citing artifact? Largely not, and we cross-check with a second judge. Controlling on the same Qwen3-32B reader and judge over 910 step-citing questions, routing the question with its step *number deleted* costs lexical selection only -0.2 pp ($28.6 \rightarrow 28.4$)—so selection locates the evidence by content, not by the index—and a different-family judge (Llama-3.1-8B) re-scoring the same answers agrees (-1.0 pp), so this is not a judge artifact. The cited step is informative *when present*: pinning it alone reaches 38.5% vs 28.6% for lexical top-5 (and 40.1 vs 36.0 under the Llama judge—ordering preserved), because it is the correct evidence, not a numeric shortcut. Corrupting the resolved address completes the picture: shifting every pin three steps off its target leaves accuracy unchanged (0.630 vs 0.629, same batch)—the near-miss keeps the referenced neighborhood in context and content routing independently recovers the exact step—while replacing every pin with a far-random wrong step costs -1.8 pp (0.611), roughly the pin's own contribution forgone, with content routing holding the floor. Structural address resolution is thus a cheap redundant guarantee that degrades gracefully under corruption, not a fragile dependency. We are careful about one boundary: non-step questions remain answerable but are genuinely harder,

and by how much is judge-dependent (within -2.4 pp of step-citing under the Qwen judge, but -14.8 pp under Llama), so we claim robustness to the step *cue*—not that non-step questions are equally easy.

Selected-context budget sweep. Within the *routed* context, more is monotonically better up to the window-forced cap: sweeping the assembled-context budget at fixed routing and reader gives 0.550 (6k tokens) $\rightarrow 0.624$ (14k) $\rightarrow 0.644$ (22k) on a 1,248-QA half (same batch; same-day replicate noise 0.5 pp). "Context rot" does not bite *selected* context at these scales—it bites unselected history (the 0.52 no-memory row of Table 3; OPENWORLD, where recency-truncated full context scores 0.474 vs routed 0.629; and the same-batch control of Table 7, where the raw trajectory recency-truncated to the *same* 22k budget costs -8.4 pp, a larger loss than any payload degradation)—so the rule is: select first, then fill the window with what was selected. The recency+gist overview earns its tokens (cutting to 6k costs -27 pp on EMBODIED and -23 pp on WEB); the one exception is TEXT2SQL, which *prefers* compact context ($+1$–3 pp at smaller budgets), pointing to budget-adaptive routing as an open lever. The converse ablation removes query-time selection instead: answering from the recency +gist overview *alone* (no verbatim appendix) costs -4.3 pp (0.5865 vs 0.6290, same-day)—notably still above every constructed-memory system (the *extractive* overview already beats generative construction), but the routed verbatim appendix clearly earns its keep.

H KV-SERVING CORRECTNESS

This appendix substantiates the identity used in §2, §4.4, and §4.7: position-preserving KV gather reproduces the full-prefill-then-mask oracle, so serving the selected spans from the cache is the same computation as reading them inside the full trajectory—not a re-prefill of them in isolation. Unless noted, the reader is a frozen dense Qwen3-8B.

The mask-oracle gate. The gathered KV is exactly the frozen model's key/value for the selected turns at their original positions, so sub-selection reproduces the full-prefill-then-mask oracle rather than a re-prefill of the selected text alone (whose kept tokens would never have attended the dropped context—a difference we measure as a ~ 13.5-logit gap to a compact re-prefill, $\sim 20\times$ the residual below). Against the mask oracle the first decoded token matches on every query: 100% first-token argmax on synthetic probes and on 144 real AMA-Bench QA ($24/24$ episodes), at a ~ 0.7 residual logit gap that is the usual bf16 split-vs-joint-prefill non-determinism.

Sequence-level, not just first-token. Greedily decoding 256 tokens, sub-selection and the mask oracle emit the *identical* full sequence on every query at both a 3.2k- and a 27k-token trajectory, at first-position logit-KL ≤ 0.011 to the oracle (median 0; versus ~ 0.28 to a compact re-prefill). The oracle's top-1-minus-top-2 answer margin averages 7.7 (min 0.25), so these are not knife-edge decisions.

Robustness to kernel and RoPE extension. The match holds under an *eager* attention kernel as well as SDPA ($6/6$), and under a $4\times$ YaRN RoPE extension (first-position logit-KL ≤ 0.008, argmax 5–$6/6$)—as expected, since position-preserving gather is exact for RoPE regardless of the base frequency.

End-to-end parity, compact layout. On real AMA under a consistent reader/judge, KV sub-selection and the deployed text pipeline are tied within noise (-0.7 pp, one QA of 144). A stratified run confirms this at n=576 QA over 72 episodes across five domains (episodes $\leq$ 26k tokens): the gate holds at $24/24$, and the two paths remain accuracy-tied with KV marginally ahead under both the same-model judge (0.175 vs 0.165, $+1.0$ pp) and an independent Qwen3-32B re-grade ($+0.7$ pp), at 81% first-token agreement between the two *different* computations.

End-to-end parity, deployed layout. The same holds when the KV arm serves the *deployed* layout: the recency+gist overview is prefilled once per episode as a post-trajectory block (fixed positions, per-episode cacheable), the selected spans are gathered from the trajectory cache, and only the question is fresh. On a 576-QA re-run ($\leq$ 24k, four domains) the gate is again $24/24$ and the arms are tied (0.222 vs 0.240, -1.7 pp, ± 5 pp binomial band; first-token agreement 90%). The two

arms differ in block *order*, so this is parity between the deployed text pipeline and its cache-native equivalent, not a byte-identical prompt; notably the overview lifts *both* arms by ~ 5 points, so the primitive covers the layout the headline numbers use.

Cross-backbone. Re-running the harness on **Llama-3.1-8B-Instruct**—a different architecture and vendor—reproduces the guarantees: first-token argmax $6/6$ across 48–200-step trajectories (residual $|\Delta\text{logit}| \leq 0.35$, tighter than Qwen's ~ 0.7) and end-to-end match with the text re-prefill on all 6 probes.

I JUDGE ROBUSTNESS

Our AMA-Bench reader is Qwen3-32B and the official harness scores it with a Qwen3-32B judge, which raises the natural worry that a same-family judge inflates the score. Two checks bound it.

First, the headline reproduces. A faithful re-run of the full official harness scores 0.643 under the Qwen3-32B judge (95% bootstrap CI $[0.623, 0.662]$ over the $2{,}496$ QA), within noise of the leaderboard-verified 0.6478.

Second, the score survives a change of judge *family*. Re-judging that run's *identical* predictions with an independent Llama-3.1-70B judge yields 0.677 $[0.658, 0.695]$ at 84% item-level agreement—higher than the Qwen judge, so on our predictions the Qwen3-32B judge is the *stricter* of the two, not the more lenient. This is consistent with the official AMA-Bench documentation, which reports a leniency bias for Qwen3-32B as a judge in *absolute* terms: because every leaderboard system is scored by that same judge, our *comparisons* sit on equal footing regardless of the absolute level. We treat the cross-family re-judge as supportive rather than definitive—it removes the same-family-inflation worry for our predictions without our needing the absolute number to be judge-invariant.

J LOCOMO: PROTOCOL NOTES

We position LOCOMO as an *out-of-domain* check rather than a headline claim, and record the caveats here. Under the public gpt-4o-mini judge, full-history prompting scores 0.756—above VeraKV's 0.704—while under a consistent Qwen3-32B judge the comparison flips ($0.716 > 0.661$); the two judges disagree on exactly this full-vs-selection comparison, which is why we report both and treat the consistent-judge result as the cleaner one (a full-history prompt is not an oracle: it carries the distractors that hurt the reader). The benchmark also has known protocol variants (QA subsets, judge choices), and newer file- and tool-based systems report stronger numbers under their own setups (e.g. filesystem-backed agents at 0.740 (Letta Team, 2025) and 0.744 (Pam Research, 2025)). The signal we draw is narrower and survives these caveats: under one fixed protocol, compact verbatim selection is competitive with dedicated dialogue-memory systems and, under a consistent judge, beats prompting the full history.